# Designing Algorithms

# 1.2

- Apply computational thinking and algorithmic design by defining the key features of standard algorithms, including sequence, selection, iteration and identifying data that should be stored
- Apply divide and conquer and backtracking as algorithmic design strategies
- Develop structured algorithms using pseudocode and flowcharts, including the use of subprograms
- Use modelling tools including structure charts, abstraction and refinement diagrams to support top-down and bottom-up design
- Analyse the logic and structure of written algorithms including:
  - Determining inputs and outputs
  - Determining the purpose of the algorithm
  - Desk checking and peer checking
  - Determining connections of written algorithms to other subroutines or functions
- Identify procedures and functions in an algorithm
- Experiment with object-oriented programming, imperative, logic and functional programming paradigms

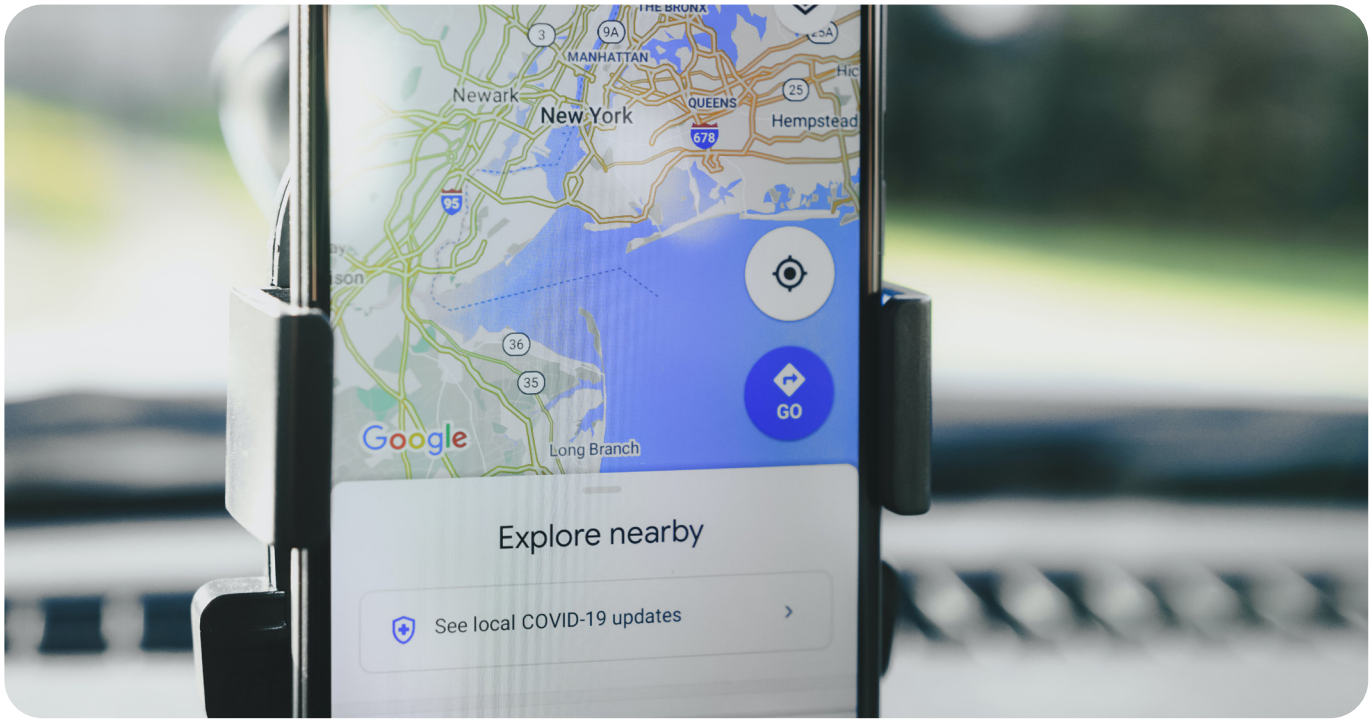
## What are algorithms?

At the core of the software engineering discipline lies a fundamental concept known as algorithms. Like a recipe guiding a chef how to make a meal, a map application directing a tourist how to reach a destination, a step-by-step tutorial for creating an animated GIF, or an instruction booklet providing the order to assemble furniture, algorithms are step-by-step procedures designed to tackle specific tasks.

Algorithms are like the recipe and method of computer programs. Algorithms streamline the language used to define what a computer program can and must do to achieve their purpose as they are based upon logic. That is, algorithms appear as simple statements and rules that outline the series of processing steps a coded program must perform.

**Key Term: Algorithm**
An algorithm is a precisely defined, step-by-step procedure or set of instructions designed to solve a specific problem or accomplish a particular task. An algorithm must have a finite number of steps which eventually comes to an end.

By using logic, software engineers establish the flow of operations, determine conditions and rules for decision-making, and ensure the consistency and correctness of the program's behaviour. By employing logical thinking and logical structures, software engineers can create reliable and efficient software solutions that solve problems and meet user requirements. This method of thinking is usually referred to as computational thinking or algorithmic thinking because it uses logic to consider solutions to problems as a series of smaller steps in a way that a computer might be able to solve them.



**Research: Algorithm Examples**
Find two examples of algorithms that power tasks such as disaster modelling, weather predictions, encryption, biometric security, song matching, data mining or natural language processing. Summarise the processing each algorithm performs.

## Computational Thinking

Computational Thinking is a problem-solving approach and a mindset that involves breaking down complex problems into smaller, more manageable parts, and then using computational techniques to analyse and solve them. It is a fundamental skill in today's digital age, applicable not only to computer science and programming but also to various real-world scenarios. Developed by computer scientists and educators, this concept has gained widespread recognition as a critical skill for individuals in the 21st century.
The key components of Computational Thinking can be summarized as follows:

### Decomposition or refinement

Decomposition involves breaking down or refining a complex problem into smaller, more manageable parts or sub-problems. This process helps in simplifying the overall problem and makes it easier to understand and solve. Dataflow diagrams and structure charts are examples of tools used to model the decomposition and refinement of problems to be solved by software engineers.

We will consider a number of decomposition and refinement strategies within this and later chapters, including top-down and bottom-up design, divide and conquer and backtracking strategies.

### Abstraction

Abstraction involves focusing on the essential details while ignoring irrelevant information. It allows one to create a simplified representation of a problem or system, making it easier to understand and work with. In programming, abstraction is evident when creating functions or classes to encapsulate complex processes into manageable components. Each of these smaller problems are solved independently.

Once the part is complete and tested thoroughly it can be used with confidence, without the need to understand the internal processing and data. A user of the part simply knows the inputs required will result in the correct outputs. Think of abstraction as hiding all the messy details.

Driving a car is a good example of abstraction – there are numerous others. There are a limited number of inputs required to drive, essentially steering, brake and accelerator which result in the output of the car driving to it's destination. This is an abstraction, as it hides all the detail of the mechanical and electrical detail that makes the car operate. Such details are ignored by the driver whose task is to focus on driving the car by knowing the effect or outputs that result from their steering, brake and accelerator inputs.

**Key Term: Abstraction**
Abstraction is the process of simplifying complex systems by focusing on essential features and hiding unnecessary details, allowing for a more manageable and understandable representation.

BRAINSTORM

**Brainstorm: Devices as examples of Abstraction**
Brainstorm a list of devices that you use daily. Explain how your use of the device is an example of abstraction.

## Algorithm Design

Algorithm design involves creating a step-by-step set of instructions or rules to solve a particular problem. These algorithms serve as the blueprint for solving problems systematically. Whether it's cooking a meal or sorting a list of numbers, algorithms provide a structured approach to problem-solving. In this chapter our main focus is on the design of algorithms.

CLASS ACTIVITY

**Activity: Students algorithm**
Each morning most high school students need to make decisions about their day. The steps taking place may include the following:

1. Wake up
2. If you are sick then stay in bed.
3. If it is a school day then continue otherwise stay in bed.
4. Get out of bed.
5. Have a shower.
6. Get dressed in school uniform.
7. Eat breakfast.
8. Pack school bag.
9. If mum is home then scab a lift otherwise catch bus to school.

There are problems with the above steps if we work through them in the precise order indicated. If you are sick, then step 2 directs you to stay in bed. We then reach step three, which indicates that on school days we must complete the steps that follow. This seems to contradict step 2's direction. This algorithm has three exit points, at steps 2, 3 and 9. It would be better if we could restructure our algorithm to have a single exit. Structured algorithms require a single exit point and, as we shall see, this is always possible.

**Problem:** Can you rewrite the steps 1 to 9 from above in such a way that there is only one exit point from the algorithm? Ensure that your new algorithm performs the tasks in the manner intended in the original.

**Research: Computational thinking in other disciplines**
Investigate other disciplines that use computational thinking. Outline at least two examples.

## Methods of Representing Algorithms

There are many methods for representing algorithms, some have very strict rules, whilst others are more English like. In this course two methods are specified quite rigorously within the HSC Course Specifications for Software Engineering, namely Pseudocode and Flowcharts.

### Pseudocode

Pseudocode is a method of describing the logic in an algorithm using a combination of natural language and code-like elements. It serves as an intermediate planning step between human-readable descriptions and the creation of actual programming code. Pseudocode employs capitalised keywords and indentation to illustrate control structures, such as sequence, selection, and iteration, used in the algorithm.

The purpose of pseudocode is to provide a high-level overview of the algorithm's logic and structure, making it easier for developers to understand and communicate the intended steps without getting into the specifics of a particular programming language. It allows developers to focus on the problem-solving and data processing aspect rather than getting caught up in language-specific syntax details. For this reason, pseudocode is written with generic wording that can be adapted to suit many programming paradigms and coding languages.

By using capitalised keywords like "IF," "ELSE," "WHILE" and "FOR," pseudocode provides a way to represent common control structures found in many programming languages. These keywords help express conditional statements, loops, and other control flow constructs without being tied to any specific programming language syntax. Indentation is used to indicate the nesting of control structures, making the code-like representation more visually structured and readable.

For consistency, pseudocode representations of control structures come in pairs. For example, for every BEGIN there is an END, for every IF there is an ENDIF. The BEGIN and END can be followed by a function name to designate a function definition for when the pseudocode is implemented into source code.

### Flowcharts

There are different types of flowcharts that are used in various fields and industries to visually communicate processes. The field of software engineering utilises several forms of flowchart. In this course flowcharts are visual diagrams that represent algorithms.

They are read from top to bottom and left to right, following the direction of arrows that connect different shapes representing data inputs and outputs, as well as the decisions and conditional statements that determine the logic flow. By combining symbols, flowcharts represent control structures like keyword pairs in pseudocode.

In software development, flowcharts serve multiple purposes. They are employed during the initial planning stages, throughout the development process, and for documentation during maintenance. Flowcharts provide a clear and concise representation of the program's flow, facilitating improved discussions between developers when planning, comparing, and reviewing solutions. Additionally, flowcharts play a crucial role in error handling and debugging as they allow developers to quickly trace the program's flow and identify potential issues. Moreover, flowcharts aid in analysing and optimising algorithms by visualising the flow and highlighting areas for improvement.


## CLASS DISCUSSION

**Discuss: Pseudocode and flowcharts**
Pseudocode and flowcharts are used to represent exactly the same thing. For example, Fig 1.2.1 and Fig 1.2.2 represent exactly the same algorithms. Which one is used, is purely personal preference. Some would argue that pseudocode is easier as it is closer to programming code. Others feel flowcharts are more visual and hence easier to construct and understand. In this course, it is necessary to be comfortable writing and reading both pseudocode and flowcharts.

**Pseudocode: Mainprogram**

```
BEGIN MAINPROGRAM
      Get Number
      Set Count to 1
      WHILE Number > Count
          Set Product to Number * Count
          Display Product
          Increment Count
      ENDWHILE
END MAINPROGRAM
```

*Fig 1.2.1*

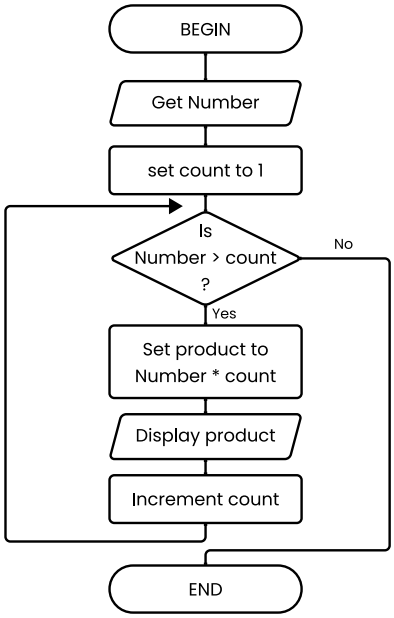



*Fig 1.2.2*

CLASS DISCUSSION

**Discuss: Pseudocode vs flowcharts**
Compare the pseudocode and flowchart shown in Fig 1.2.1 and Fig 1.2.2. Can you see how both algorithms are describing the same method of solution? Discuss.

**Discuss: Pseudocode vs flowcharts**
What problem do you think is being solved by the algorithms in Fig 1.2.1 and Fig 1.2.2? Describe, in words, the operation of the algorithms that lead to the solution of this problem.

## Control Structures

Structured algorithms are built using control structures. This process is known as structured programming. Control structures determine the direction or order in which statements within an algorithm are executed. Control is the influence that directs the flow of execution. Control structures are standard constructs used when creating structured algorithms.

The theory behind structured programming is that all problems can be solved using just three different control structures; sequence, selection and iteration. This theory has never been definitively proven however no problem has ever been found that cannot be solved using structured techniques. All imperative and procedural programming languages include statements provide different implementations of each of these control structures as part of software solutions. Writing algorithms in pseudocode or as a flowchart means the same algorithm can be implemented in virtually any programming language – the algorithm is independent of the programming language.

Although only the three control structures; sequence, selection and iteration are required, a fourth, the use of subroutines or subprograms, is desirable when decomposing problems using top-down design development techniques.

A fifth, known as recursion, is a further control structure where a subprogram (usually in the form of a function) calls itself from within. Recursion simplifies many problems and hence is an important and elegant technique to master. We will introduce examples of recursion later in this chapter, and you will see examples of recursion used again as we introduce data structures in chapter 1.3.

In this section we examine the three control structures essential to structured programming, namely sequence, selection and iteration.

## Sequence

In structured programming the order of processing is important. Each process must be performed in the correct order for the solution to be realised. Sequence is the control structure that ensures each process occurs in the correct order. For example, when getting dressed you must put your socks on before your shoes.

**Sequence pseudocode**

```
BEGIN
    Process 1
    Process 2
    Process 3
    Process 4
END
```

**Sequence flowchart**

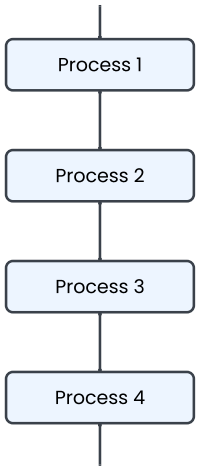



*Fig 1.2.3*

There are programming languages where the sequence of processing is not significant. Consider a spreadsheet; formulas are entered into cells with little need to consider the order in which these formulas will be evaluated.

When using pseudocode each process is written one under the other. Similarly on flowcharts the order of processing moves from top to bottom. Each process is completed before the next is commenced. A single process that is out of order will most likely effect the operation of the entire algorithm.

### CLASS ACTIVITY

**Discuss: Leaving garage sequence**

To back a car out of a garage requires the following steps: start the car, open the garage door, get in the car, release the hand brake, engage reverse gear, press accelerator, check review mirror, unlock car and place foot on brake.

The steps above are obviously not in the correct sequence. Rewrite these steps in the correct sequence first as a flowchart and then in pseudocode. Is there only one possible sequence?

**Consider: Swapping sequence**

Many software solutions require that the contents of two variables need to be swapped. A precise sequence of events must be adhered to if this process is to occur correctly. The algorithms below are attempting to swap the data item in Num1 with the data item in Num2.

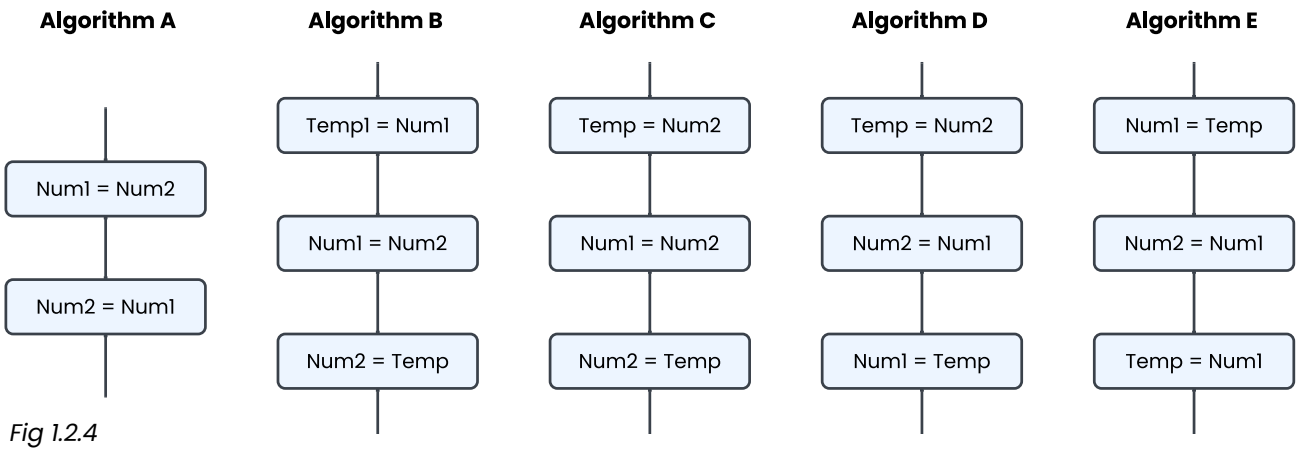



*Fig 1.2.4*

CLASS ACTIVITY

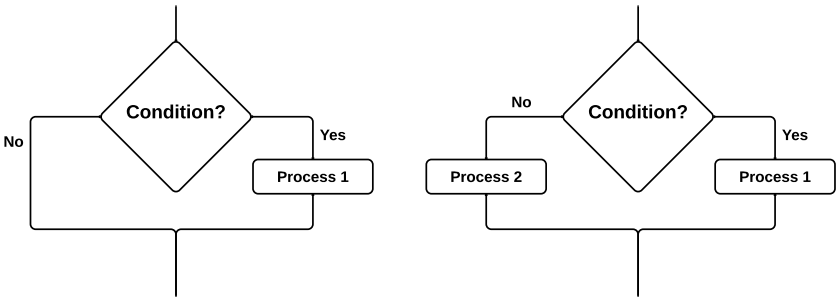
**Discuss: Swapping algorithm**
Some of the algorithms above correctly perform the swap while others do not. Identify the correct and incorrect algorithms. Explain your response for each algorithm. Discuss.

## Selection

Selection is the control structure that allows decisions to be made between different alternative paths. Different paths are executed in response to the outcome of some condition. Once the processes along this path are complete processing continues with the statement following the selection control structure.

### Binary selection

In binary selection there are two alternatives, one being selected if the condition is true and the other if the condition is false. The condition results in a Boolean value, either true or false. Often one branch will not involve any additional processes, rather its purpose is to avoid execution of the processes present on the alternate branch. Fig 1.2.5 shows the syntax used to represent binary selection using pseudocode and flowcharts.




```
IF Condition THEN
    Process 1
ENDIF
```

```
IF Condition THEN
    Process 1
ELSE
    Process 2
ENDIF
```

*Fig 1.2.5*

When using binary selection, the condition can be expressed as a statement e.g. Number>2 in which case the result is either True or False. It can also be expressed as a question e.g. Is Number>2? in which case the answer is either Yes or No. In either case, it is important on flowcharts to label each branch appropriately using Yes/No or True/False. In this text we prefer to use questions combined with Yes and No.

CLASS ACTIVITY

**Discuss: Selection control structure**
Under current Australian law you must be 18 years of age to vote. An algorithm to determine if someone can vote is described in Fig 1.2.6 as a flowchart and in pseudocode.

The flowchart shown in Fig 1.2.6 uses the three symbols. Describe the purpose of each symbol in terms of making flowcharts more understandable.

```
Get Age
IF Age ≥ 18 THEN
    You can vote
ELSE
    You can't vote
ENDIF
```

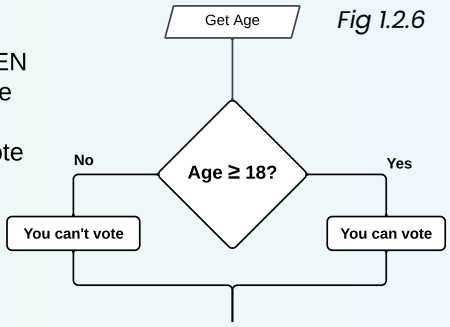



*Fig 1.2.6*

## Multiway selection

Multiway selection caters for situations where more than two alternative paths are required. The first choice encountered that makes the expression true causes control to branch to that respective path. Multiway or multiple selection statements can be implemented using multiple binary selection statements. Multiway selection statements are not a necessary part of structured algorithms however in many instances they greatly reduce the length and complexity of the solution.

```
CASEWHERE Expression is
    Choice A   :Process 1
    Choice B   :Process 2
    Choice C   :Process 3
    Choice D   :Process 4
       .           .
       .           .
       .           .
    Otherwise   :Process n
ENDCASE
```

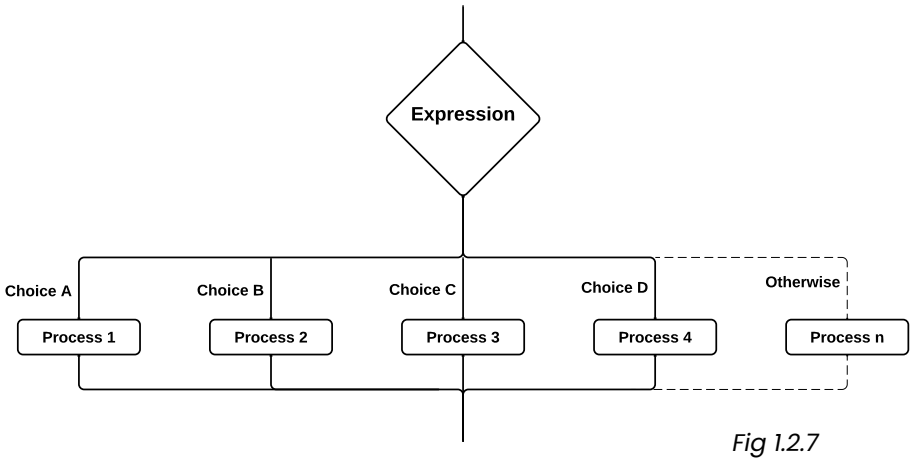



*Fig 1.2.7*

The `Otherwise` choice is a default should none of the available choices make the expression true. It can be likened to the `No` or `Else` part of the binary selection structure. It is not necessary to include the `Otherwise` choice in all algorithms.

### CLASS ACTIVITY

**Discuss: Multiway selection algorithm**

A triathlon is being organised. As part of the administration of the event, a software application is developed to sort the entrants into their respective levels. Competitors who are 12 years old or younger are juniors, 15 years or under are intermediate, younger than 18 are seniors. Competitors who are 35 years or older compete in the veteran level and the remaining competitors are placed in the open level. An algorithm for this aspect of the software product has been developed:

```
CASEWHERE Age is
            ≤ 12   :Junior
            ≤ 15   :Intermediate
            < 18   :Senior
            ≥ 35   :Veteran
    Otherwise   :Open
ENDCASE
```

The algorithm in fig 1.2.8 has been created using multiway selection. Rewrite the algorithm using only binary selection structures.
Do this as a flowchart and then in pseudocode.

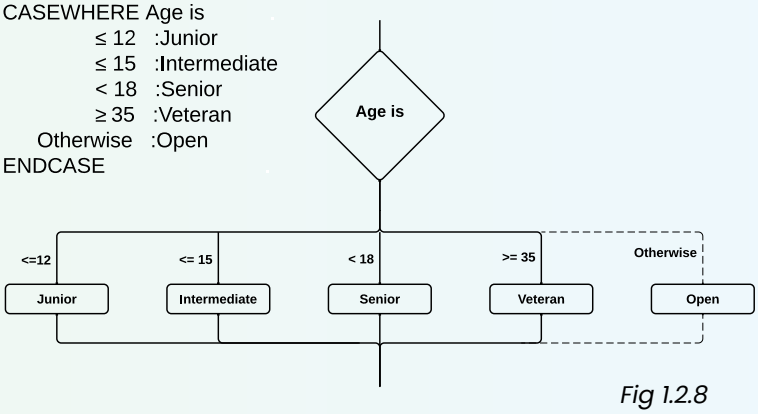



*Fig 1.2.8*

## Iteration (or Repetition)

Iteration is the repetition of a sequence of steps. A termination condition is used either at the start or at the end of the sequence to stop the repetition. Iteration is often called looping as control passes from the last statement in the sequence back to the first forming a loop. The processes within the loop structure are known as the body of the loop.

Program runtime errors are often a consequence of poor iteration structures where the termination condition is never met. This results in the appearance, to the user that the computer has frozen. In actuality it is processing at full speed but is stuck in an infinite loop.

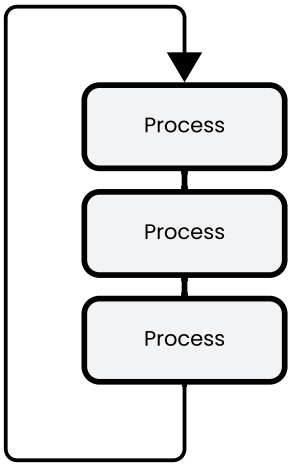


*Fig 1.2.9*

Iteration is what makes computers such powerful calculating machines. Their ability to repeat instructions continually at incredible speeds makes the software applications we use each day possible. For example, the animation sequences in computer games, the screen redrawing millions of colours some 70 times per second, and weather forecasting applications processing vastamounts of data.

There are two main types of iteration; pre-test where the termination condition is at the start of the loop and post-test where the termination condition follows the body of the loop. A third structure known as a counting loop or for…next loop is a special case of the pre-test loop that is used so often that it is included in most programming languages.

### Pre-test iteration

Pre-test iteration is the most commonly used form of repetition. Because the termination condition comes before the body of the loop it is possible that the processes within the loop will not be executed at all. For this reason, pre-test loops are also known as guarded loops, the condition guards the entrance to the loop. Most algorithms require the possibility of not entering the loop if all possibilities are to be taken into account.

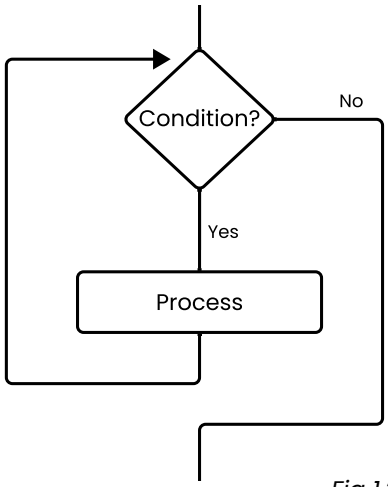


*Fig 1.2.10*

With pre-test loops the repetition continues whilst the condition remains true. It is when the condition becomes false that the repetition terminates. As a consequence, it is essential that the processing occurring within the loop alters the contents of variables that form the condition. If this does not occur the loop will continue infinitely.

The pseudocode implementation of pre-test iteration uses the keyword pair WHILE and ENDWHILE. The initial WHILE is followed by a condition, which must remain true for the repetition to continue. ENDWHILE signifies the end of the loop and that control should return to the WHILE statement. On flowcharts, the ENDWHILE keyword is replaced by a flow line connecting back to above the condition. This flow line requires an arrow as it directs control in an upward direction rather than the normal downwards direction.

CLASS ACTIVITY

**Discuss: Pencil algorithm**

To sharpen a pencil you continue turning the sharpener whilst the pencil is not sharp. Fig 1.2.11 shows this algorithm expressed in pseudocode and as a flowchart. The use of pre-test iteration means that if the pencil is already sharp we don't ever turn the sharpener.

```
WHILE Pencil is blunt
    Turn the sharpener
ENDWHILE
```

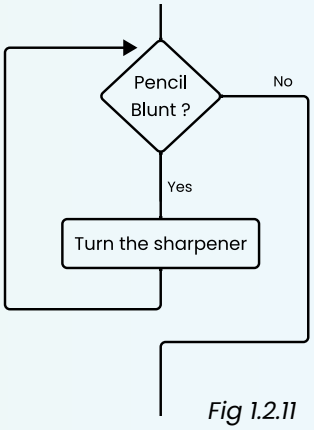



*Fig 1.2.11*

Pre-test iteration continues whilst the condition is true. The condition iscontained within a decision diamond. What makes this decision different to that used for binary selection? Discuss.

## Post-test iteration

Post-test iteration is where the termination condition follows the body of the loop. This means that the processes within the body of the loop are always executed at least once. For this reason post-test loops are also called unguarded loops. When using a post-test loop you must be absolutely certain that under all circumstances the processes within the loop should occur at least once.

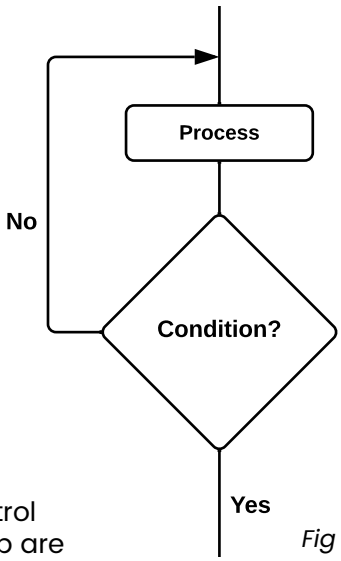



*Fig 1.2.12*

Checking user input is a common situation where a post-test loop is appropriate. The body of the loop gets input from the user. The termination condition checks that the input is reasonable. If it is not then further input is requested. If the input is successfully validated then the loop is terminated.

Consider the representations shown in Fig 1.2.12 On the flowchart control enters the body of the loop from above. The processes within the loop are executed then the condition is checked. If the condition is true the loop terminates, if not then control returns to the top of the loop. In pseudocode the keyword REPEAT is synonymous with the arrow on the flowchart leading back up to the top of the loop. Effectively the word REPEAT is merely a marker indicating the top of the loop. Similarly, UNTIL indicates the end of the iteration and has the same meaning as the decision diamond on the flowchart. Remember, both the pseudocode and the flowchart are representing exactly the same control structure.

## FOR loops

For loops are a special case of pre-test iteration. These loops are used when we wish to repeat a sequence of processes a set number of times or we wish repetition to occur whilst a value is incrementing within the limits of two values. For example, if we require a variable to take the values 1, 2, 3, 4, 5… 100 then a for loop is the most convenient structure to use.

In most programming languages it is possible to alter the magnitude of each increment (or step) to produce sequences such as 10, 9, 8, 7, … 0 or even -1, -0.5, 0, 0.5, 1, 1.5… 10. The incrementing value, called the loop counter, changes by a set amount (the step) each time the body of the loop has been executed. Counting loops are so often required that most programming languages include them as a standard construct.

Strictly speaking, we do not require a specific method for representing for loops using our two methods of algorithm description, as the pre-test iteration control structure can be used to more accurately describe the logic. However for convenience, the FOR…NEXT keywords can be used in pseudocode. If a flowchart is being used then the explicit logic of the loop should be shown.

**Example: For loop suggested syntax (pseudocode)**

```
FOR LoopCounter = InitialValue TO FinalValue STEP StepValue
      Process
NEXT LoopCounter
```

*Fig 1.2.13*

The pseudocode in Fig 1.2.13 represents a counting iteration structure. During the first iteration, the loop counter holds the initial value. During the second iteration the loop counter is equal to the initial value plus the step value. Subsequent iteration increment the loop counter by the step value. The loop terminates once the loop counter reaches a value greater than the final value if the step value is positive. If the step value is negative, then the loop terminates once the loop counter is less than the final value. Be careful when using fractional step values, as the approximate nature of floating point representations can cause the loop to terminate earlier or later than expected.

### CLASS ACTIVITY

**Activity: FOR loop algorithm**
In two programming languages of your choice, using an online editor/interpreter, develop an algorithm that displays the first 10 multiples of a number entered. For example, if the user enters 3 then the algorithm should display 3, 6, 9, 12, 15, 18, 21, 24, 27 and 30.

A possible solution is described in Fig 1.2.14. Notice that the STEP part of the pseudocode is not required when the step value is 1.

**Pseudocode: For loop**

```
Get Number
FOR Count = 1 TO 10
      Multiple = Count * Number
      Display Multiple
NEXT Count
```

*Fig 1.2.14*

The logic is clearer if we write algorithms that include counting loops using pre-test iteration structures rather than the FOR…NEXT pseudocode keyword pairs, however this does require a little more effort. Suppose a problem requires a loop that outputs the even numbers from 2 to 100. Let us develop this algorithm using pseudocode and a counting loop, pre-test pseudocode and finally as a flowchart. Each of these algorithms is logically identically, it is the method used to describe them that is different.

**Pseudocode: For loop**

```
FOR Count = 2 TO 100
      Display Count
NEXT Count
```

**Pseudocode: While loop**

```
Count = 2
WHILE Count <= 100
      Display Count
      Count = Count + 1
ENDWHILE
```

*Fig 1.2.15*

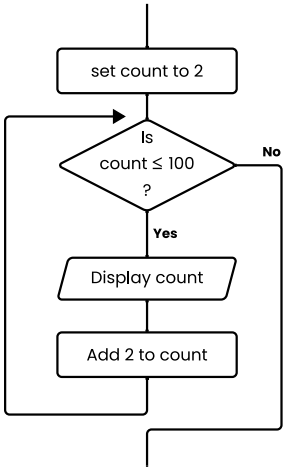



*Fig 1.2.16*

The FOR statement in the FOR…NEXT version includes setting the initial value of the loop together with the decision. This corresponds to the first two lines of the pre-test pseudocode version. The NEXT Count statement corresponds to the last two lines of the second version. The flowchart is a precise copy of the second pseudocode with minor changes to the wording of some statements.

There is a difference in the way counting loops are implemented in some programming languages that is not immediately obvious. When the loops within the pre-test lopp algorithms above terminate, Count will have a value of 101. In most programming languages, this is also the case when the FOR loop version is implemented. Unfortunately it is not always the case. Sometimes Count will be undefined, and sometimes it will be the final value given in the FOR statement. It is important to understand the intricacies of the FOR…NEXT implementation in the programming language you are using.

## CLASS DISCUSSION

**Discuss: Loop analysis**

Study each of the algorithms in Fig 1.2.15. Describe how each of these algorithms operates to perform the stated task

Describe how each of the above algorithms could be changed to display the following sets of numbers:

**A:** 1, 2, 3, 4, 5,…50.

**B:** 100, 95, 90, 85,… 0.

**C:** -45, -35, -25,… 45.

## EXERCISE 1.2.1 - MULTIPLE CHOICE

1. What is the control structure that allows decisions to be made between alternative paths?

   A. Sequence.
   B. Iteration.
   C. Selection.
   D. Subroutines.

2. The most commonly used form of repetition is:

   A. pre-test iteration.
   B. post-test iteration.
   C. iteration.
   D. none of the above.

3. Keywords used in pairs, are a characteristic of which method of representing an algorithm?

   A. Flowcharts.
   B. Context diagrams.
   C. Pseudocode.
   D. None of the above.

4. Which shape is used on flowcharts to represent input and output?

   A. Rectangles.
   B. Diamonds.
   C. Squares.
   D. Parallelograms

5. When using binary selection, how many alternatives are there?

   A. One.
   B. Two.
   C. Eight.
   D. Sixteen.

6. What determines the order in which statements within an algorithm are executed?

   A. Subroutines.
   B. Control structures.
   C. Parameters
   D. Variables

7. Kate is a SDD student who has been given the task of representing the same algorithm using two different methods. Which two common methods for doing this, are examined in this course?

   A. NS diagrams and structured English.
   B. Pascal and Cobol.
   C. Context diagrams and decision trees.
   D. Pseudocode and flowcharts.

8. Post-test loops are also known as:

   A. counting loops.
   B. guarded loops.
   C. unguarded loops.
   D. none of the above.

9. Which shape is used on flowcharts to represent decisions?

   A. Diamonds.
   B. Squares.
   C. Rectangles.
   D. Circles.

10. What is the control structure that ensures that each process is executed in the correct order?

    A. Iteration.
    B. Sequence.
    C. Selection.
    D. Pre-test iteration.

## EXERCISE 1.2.1 - SHORT ANSWER

11. What is an algorithm and why are they so important when designing software solutions?

12. What is structured programming? Discuss.

13. Explain the operation of each of the three fundamental control structures. Use examples of each as part of your explanation.

14. Create an algorithm that calculates the average of a series of numbers.

15. Create an algorithm that describes the operation of a light that can be turned on or off using either of two switches.

## Modular Design

Modular design is an approach used in software development to break down, decompose or refine complex problems into smaller, more manageable parts called modules, objects, classes, subroutines, subproblems or subprograms depending on the nature of the problem. These parts represent self-contained units where each performs a specific task to achieve a required purpose. Once created and thoroughly tested these units can be used and reused with confidence. We can then utilise these parts without the need to understand or consider the detail of the processing occurring under the hood – we know it works - this is abstraction. By dividing the problem into smaller parts, developers can focus on one piece at a time, making the overall development process more organised, manageable and efficient.

### CLASS DISCUSSION

**Modular design**

Modular design can be compared to building with interlocking blocks, like LEGO. Just as LEGO blocks can be easily connected and combined to create various structures, modular design allows software developers to break down a complex program into smaller, manageable modules. Each module serves a specific purpose and can be constructed independently and at different development speeds, like how LEGO blocks can be individually designed and then assembled to build different creations. Modular design promotes flexibility, reusability, and scalability in software development, just like how LEGO blocks offer endless possibilities for constructing different objects.

**Brainstorm: Modular Design**
There are numerous examples where modular components are utilised to construct a range of products and solutions. List a number of examples, along with their component modular parts.

When tasked with developing solutions to problems using code, software engineers use strategies that consider problems as smaller subproblems, subprograms or modules:

- **Top-down design** is a design approach that progressively breaks a larger problem into smaller subproblems. Each of the subproblems is more manageable and can be solved in isolation to the larger problem. It emphasises breaking down problem into modular components, starting with a high-level overview and gradually refining the details. It involves identifying the main functionalities and designing the overall structure of the system or whole software application first. Then, each component is further broken down into smaller, more detailed modules or functions.

- **Bottom-up design** focuses on building a system or solution from smaller, foundational components or modules, and gradually combining them to create a complete and functional system. These modules are self-contained and can function independently in that they are designed to perform specific tasks or provide certain functionalities. Once the modules are developed and tested, they are integrated and combined to form larger, more complex components, eventually leading to the creation of the complete system.

- **Divide and conquer** involves the use of recursive calls to progressively break down a complex problem into smaller, more manageable subproblems. Each subproblem is a similar, yet simpler problem to the one just above. The approach works by recursively solving these subproblems independently and then combining their solutions to obtain the final solution. By dividing the problem into smaller versions of the larger problem, the complexity of the overall problem is reduced, making it easier to solve. We will examine Merge Sort in detail as an example of divide and conquer.

- **Backtracking** involves systematically exploring different options to find a solution. It works by trying out possibilities and, if a chosen path leads to a dead end or incorrect solution, it goes back and tries a different path. Backtracking algorithms navigate through many possibilities by discarding unpromising choices early on. It is commonly used when searching for solutions amongst a large number of possible solutions. Often such problems involve combinations or permutations together with specific constraints or requirements that need to be satisfied. We will examine the N-Queens problem as an example of a backtracking algorithm.

Modular design or 'modularity' brings many advantages to software development:

1. **Code reusability**: Modular design promotes code reusability. Once a module is developed and tested, it can be used in multiple parts of a software system or even in different projects. This reduces redundant coding efforts, saves time, and improves productivity.
2. **Ease of maintenance**: When a software system is modular, it becomes easier to identify and fix issues or make enhancements. Since modules are independent and self-contained, changes made to one module are less likely to affect other modules. This simplifies the debugging and maintenance process.
3. **Enhanced collaboration**: Modular design facilitates collaboration among team members. Developers can work on different modules simultaneously, independently of one another. This parallel development approach speeds up the overall development process and allows team members to specialise in specific areas.
4. **Scalability**: Modular design enables scalability, which means that additional modules can be added or modified later to accommodate changes or future requirements. New features can be integrated by developing new modules or extending existing ones, without impacting the entire system.
5. **Improved testing**: With modular design, it is easier to test individual modules independently. This ensures that each module performs its intended function correctly. Isolated testing of modules simplifies the identification of issues and speeds up the debugging process.
6. **Code readability**: Modules promote code readability and understanding by making the overall code more organised and easier to comprehend. This makes it easier for developers to navigate, maintain and update the software system over time.

## Abstraction and refinement diagrams

Abstraction and refinement diagrams (also known as hierarchy charts) are a simple method used to represent the modular design top-down or bottom-up design of a program. Modules or subroutines are often numbered to indicate their placement within the hierarchy of the modular design.

Structure charts can be thought of as an extended abstraction and refinement diagram. They add the three control structures to the diagram. Sequence indicated by the left to right order, selection via diamonds on the connection between subprograms, and iteration using circular arrows. Data passed between subroutines is also added.

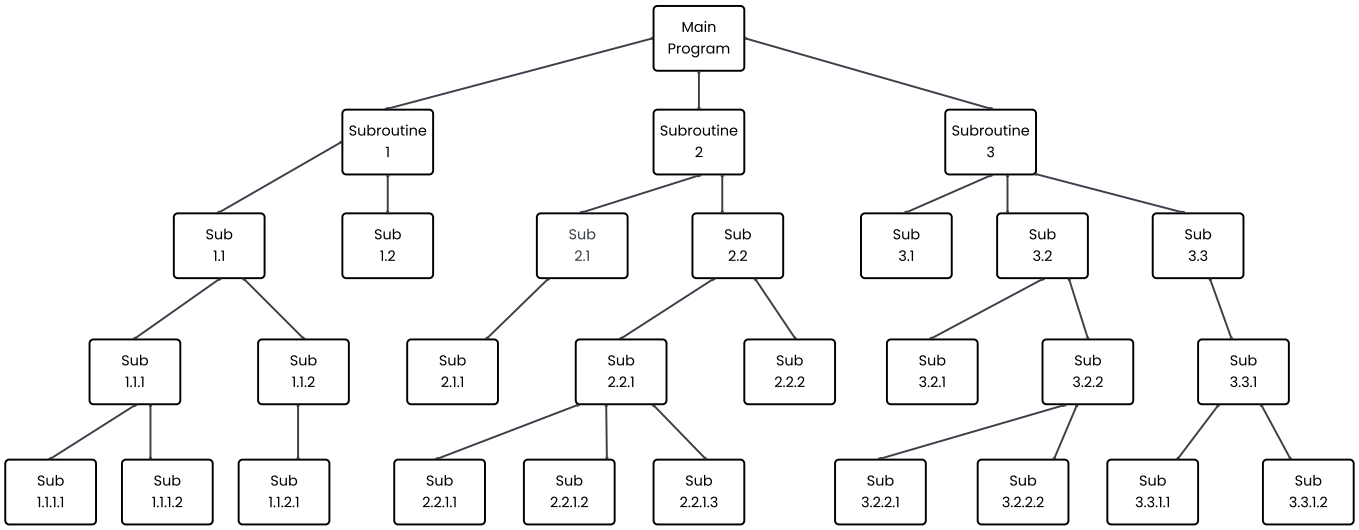



*Fig 1.2.17*

## CLASS ACTIVITY

**Discuss: Meal design**

Imagine you've decided to cook a special meal for your family. Let us develop a modular design to describe the processes involved in this venture. Firstly, you must decide on the time and the date of the meal. You must then work out the menu and obtain the ingredients. On the allocated day you must cook and serve the meal. Fig 1.2.18 describes our solution so far.

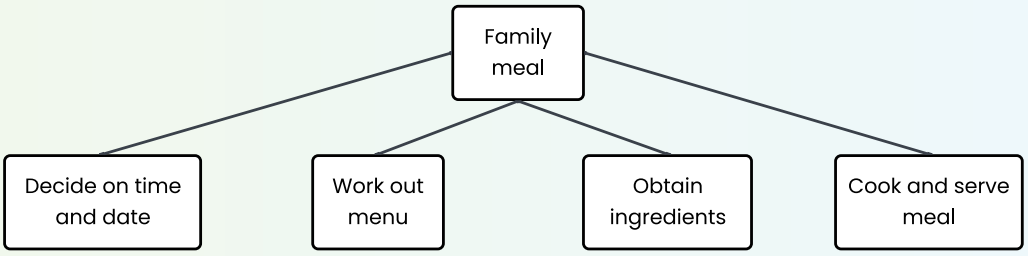



*Fig 1.2.18*

We can now consider in isolation each subroutine on level 1 of our hierarchy chart. As we consider each subroutine we ignore the larger problem – this is the process of abstraction. Deciding on the time and date involves questioning each family member and then picking the most appropriate time and date. We are refining the problem using top-down design or stepwise refinement. Working out the menu includes considering the tastes of those who'll attend and then examining recipe books. Obtaining the ingredients may include working out the quantities required, checking the pantry and then visiting various shops. Cooking and serving the meal includes preparing the ingredients, cooking them and finally serving to each family member.

Let us expand the preparation of the ingredients branch of our hierarchy chart further as shown in Fig 1.2.19. This may include rinsing the vegetables, chopping the ingredients, measuring quantities and mixing ingredients. Chopping may further involve first peeling vegetables and perhaps grating and dicing.

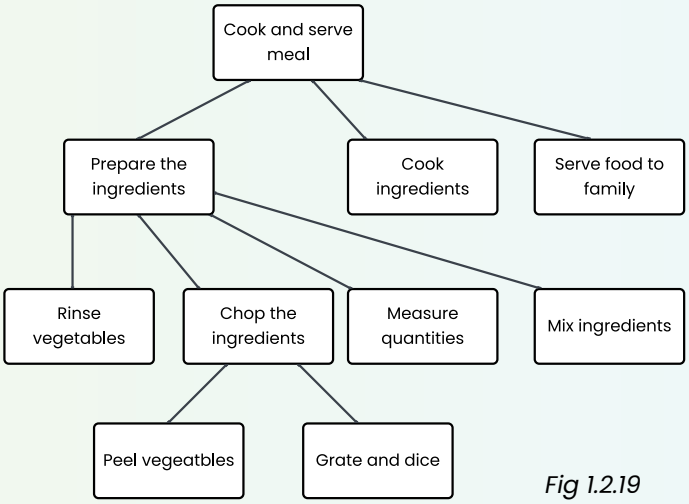



*Fig 1.2.19*

**Activity: Meal design**
Expand each branch of the initial hierarchy chart shown in Fig 1.2.18. Add a numbering system to your completed chart.

## SCENARIO & DISCUSSION

- A large multi-national software company specialises in the development and marketing of software to the retail industry. One of their largest selling software products is able to interface with various types of cash register and EFTPOS terminals. Currently there is no standard in place for these communication links. As a consequence, the company must develop a new software interface each time new models of cash register or EFTPOS terminal become available.
- A web site developer works from home. He charges an hourly rate to design and develop web sites for various clients. At present he has been in business for about 2 years and is finding that past clients are increasingly requesting modifications to their original sites. Implementing these modifications is proving difficult and time consuming.
- The Year 2000 bug was due to programs only considering and storing the last two digits of the year e.g. 1931 was stored as just 31. The time spent examining software products to ensure Year 2000 compliance was considerable. Perhaps the most annoying aspect of the problem was that most products were found to be compliant without the need for any modification.

CLASS DISCUSSION

**Discuss: Modular design**
For each scenario above, discuss how modular design, abstraction and reusable modules simplify the process of modifying the software.

**Discuss: Recycle or Recreate?**
In many cases it is just simpler to throw out the old product and develop a new one from scratch. Do you agree with this statement? Explain and discuss your answer using the above scenarios as examples.

## Subroutines

Software developed using top-down design is comprised of a series of subroutines. Subroutines are also called subprograms or procedures. The terms subroutine and subprogram imply a lower level process whereas the term procedure refers to its sequence of statements. The highest level routine is known as the main program. The main program contains calls to lower level subroutines. In turn these subroutines call lower level subroutines and so on.

When using pseudocode, a call to a subroutine is indicated by underlining the process (see Fig 1.2.20). The underline tells the reader that there is more detail in regard to this process elsewhere. Furthermore, the subroutine found elsewhere will have the same name as the underlined words. The keywords BEGIN…END together with the underlined name of the routine are used to enclose the processing of the subroutine.

```
BEGIN MAINPROGRAM
      Process 1
      Process 2
      Process 3
END MAINPROGRAM

BEGIN Process 1
      Do something
      Do something else
END Process 1

BEGIN Process 3
      Do whatever
      Do whatever else
END Process 3
```

*Fig 1.2.20*

When representing algorithms using flowcharts a call to a subroutine is indicated using additional vertical bars on either side of the process box (see Fig 1.2.21). The start of the main program commences with the word BEGIN in a box with rounded sides and ends with a similar box containing the word END. Some references use the words start and stop in preference to begin and end. The commencement and termination of a subroutine is similarly indicated with the addition of the subroutine's name within each symbol.

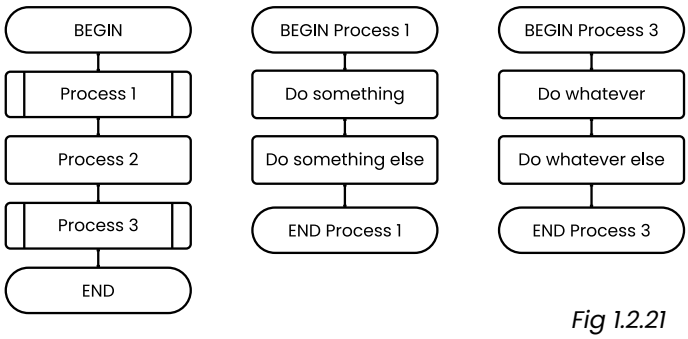



*Fig 1.2.21*

CLASS DISCUSSION

**Discuss: Modular design of software**
Fig 1.2.20 and Fig 1.2.21 describe the same algorithm. They also describe the modular design of software. Discuss how these methods of indicating subroutines assist developers during the modular design process.

## Parameters

Parameters provide the interface between different subroutines within a software product. These are the same parameters specified on structure charts and used as data flows on dataflow diagrams. They allow subroutines to access and alter the data items held in variables. These variables may be simple data types or they may be complex data structures. Higher level subroutines call lower level subroutines using actual parameters. Actual parameters are real variables that contain data. Formal parameters are used within subroutines. Formal parameters are effectively replaced by actual parameters whilst the subprogram is executing.

Let us consider how parameters are represented when using pseudocode and flowcharts. In both cases, the list of parameters to be sent and returned from the subroutine is included within a bracketed list. The order in which the parameters are listed is significant; it determines their destination when they arrive at the subroutine. In Fig 1.2.22 the actual parameter First in the call to the subroutine Biggest corresponds to the formal parameter Item1 within the subprogram. Similarly, the actual parameter Second corresponds to the formal parameter Item2 in the subprogram.

```
BEGIN MAINPROGRAM
      Get First
      Get Second
      Biggest (First, Second, Result)
      Display Result
END MAINPROGRAM

BEGIN Biggest(item1, item2, Big)
      IF item1 > item2 THEN
            Big = item1
      ELSE
            Big = item2
      ENDIF
END Biggest
```

*Fig 1.2.22*

The process of communicating via parameters is known as passing. In our Biggest example (Fig 1.2.22 and 1.2.23) the actual parameter First is passed to the formal parameter Item1 in the Biggest subprogram. In reality, the variable First is an identifier pointing to a specific address in memory. First can be passed to Item1 in one of two ways, either "by reference" or "by value":

**1.** When passing "by reference" a pointer to the address in memory of First is sent to the formal parameter Item1. In this case no new memory location is created rather Item1 will point to the memory location received from the actual parameter First. Whilst the Biggest subroutine is executing, each reference to Item1 causes the memory location associated with the formal parameter First to be accessed. It is often helpful to think of each occurrence of the identifier Item1 being replaced by the identifier First whilst the subroutine is active.

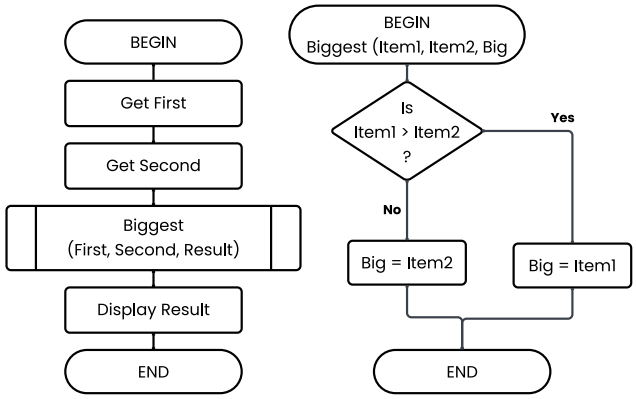



*Fig 1.2.23*

If the Biggest subroutine altered the value of Item1 during processing then the value of First in the main program will also change. This makes sense as both Item1 and First are merely different names for exactly the same memory location. In summary, passing by reference passes data in both directions – both to and from subroutines. In the Biggest subroutine the variable Big must be passed by reference as this is the variable that returns the output back to the calling routine.

**2.** When passing "by value" a copy of the value held in the variable First is sent rather than the memory address. This means there is no connection between corresponding parameters, such as First and Item1, during the processing of the Biggest subroutine. The Biggest subroutine must create memory locations for each of its parameters and store the received values in these locations. More
significantly the value of the original formal parameter is not altered during execution of the called subroutine. In summary, passing by value only passes data into subroutines. In many current programming languages passing by value is the default behaviour. This prevents changes to variables in calling subroutines from being changed unintentionally.

To further confuse the issue, in most programming languages the identifier associated with many data structures (such as arrays) store pointers to the memory address of the data structure. Therefore regardless of whether an array is passed "by reference" or "by value" to a subroutine it is the address of the location of the array in memory that is passed. This means the values within the actual array elements can be altered by the subroutine.

Many program languages, such as Java for example, pass all parameters by value. To return a value from a subroutine requires use of the RETURN keyword. In this case each subroutine is a function. And all subroutine calls are function calls. Fig 1.2.24 shows how such calls are specified in pseudocode and flowcharts. Functions receive one or more inputs and return a single output. Each unique set of inputs will always return the same output. However, different sets of inputs may well produce the same outputs. This behaviour is the same as functions in maths. For example, in Fig 1.2.24 inputs of 3,4 always return 4, however inputs of 4,2 will also always return 4.

```
BEGIN MAINPROGRAM
      Get First
      Get Second
      Result = Biggest (First, Second)
      Display Result
END MAINPROGRAM

BEGIN Biggest(item1, item2)
      IF item1 > item2 THEN
            Big = item1
      ELSE
            Big = item2
      ENDIF
      RETURN Big
END Biggest
```

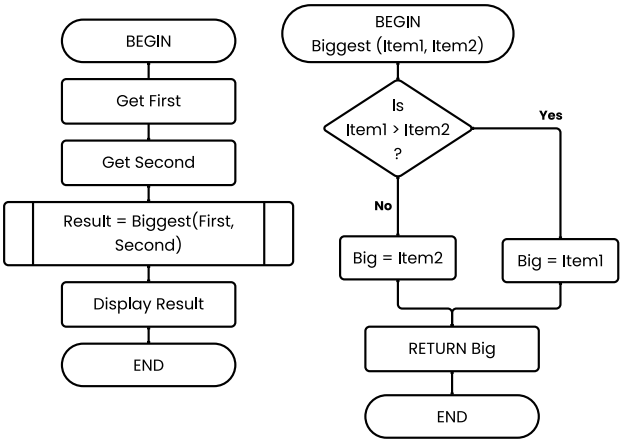



*Fig 1.2.24 pseudocode and flowchart*

**Consider the following:**

Imagine you work part time at the local library. Often you have the task of locating books for the librarian. The librarian gives you a list containing the call numbers of each required book. You then go to the shelves and retrieve these books. This is similar to the way a call where parameters are passed by reference works. You are playing the part of the subprogram and the librarian is the main program. The shelves are memory areas containing the data (the books). The call numbers represent the actual locations of the books; the actual parameters. There is only a single book with a unique call number. You receive the list of call numbers and use them as formal parameters. Whether you or the librarian find a particular book it remains the same book.

Now imagine the librarian wishes you to comment on a newspaper article. She makes a photocopy of the original article and hands it to you. You then take the photocopy to your desk and as you read you highlight significant words and ideas. You then write down your comments on a piece of paper, hand them to the librarian and toss the photocopy of the article in the bin.

This is like passing by value. Again the librarian is playing the part of the main program and you are playing the subroutine role. The article is passed by value meaning a copy of the data is passed rather than a link to the original. Clearly your highlighting will not appear on the original newspaper article as it is merely a copy of the original. Also, tossing the photocopy in the bin has no effect on the original. The return value is your comments which you (the subroutine) hand back to the librarian (the main program).

**Activity: Library scenario**
Act out the above library scenario. Pause as the drama unfolds to discuss how each component and process compares and assists your understanding of calling subroutines with parameters.

**Discuss: Usage of parameters**
The use of parameters is central to the programming process. Most commands, functions and even operators included within programming languages use parameters. Discuss using examples.

A word search is a puzzle composed of a grid of letters. The objective is to find all the words hidden in the grid. The words can run either across (horizontally) or down (vertically). In this word search, words do not run diagonally, and they always run from left to right or from top to bottom.

For example, consider the following Word Search puzzle:

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| S | U | W | Q | W | L | R | E | D | O | J | R |
| Z | T | U | V | E | C | X | S | J | R | V | P |
| B | N | B | L | A | C | K | P | G | A | W | I |
| L | B | R | O | W | N | S | Y | Y | N | H | N |
| U | N | X | M | Y | R | W | H | W | G | I | K |
| E | R | O | G | R | E | E | N | T | E | T | M |
| F | P | J | F | M | E | M | V | P | D | E | E |
| F | Y | E | L | L | O | W | V | R | N | S | A |

The following words are in the above puzzle:

BLACK, BLUE, BROWN, GREEN, ORANGE, PINK, RED, WHITE, and YELLOW.

A program already exists to generate word search puzzles like the one above.

A program to solve word search puzzles is being developed based on the following structure chart.

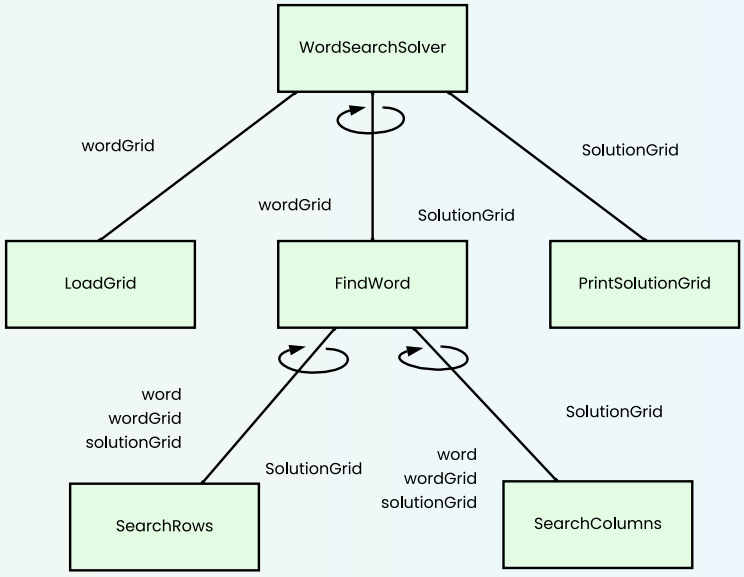



Fig 1.2.25

**Problem:**

Write an algorithm for the main program called WordSearchSolver based on the provided structure chart.

(3 marks)

*Suggested solutions top of page 95.*

**Suggested Solution:**

```
BEGIN WordSearchSolver
        LoadGrid(wordGrid)
        WHILE more words
                FindWord(wordGrid, solutionGrid)
        END WHILE
        PrintSolutionGrid(solutionGrid)
END WordSearchSolver
```

**Alternative Solution:**

```
BEGIN WordSearchSolver
        wordGrid = LoadGrid
        WHILE more words
                solutionGrid = FindWord(wordGrid)
        END WHILE
        PrintSolutionGrid(solutionGrid)
END WordSearchSolver
```

*Fig 1.2.26*

**Arrays**

The concept of an array is fundamental in computer science and programming, providing a way to organise and manage data in a systematic manner.

An array is a data structure that stores a collection of elements, each identified by an index or a key. The index is an integer commencing at 0 through to the number of elements in the array – 1. For example, an array containing 5 elements would be indexed from 0 to 4.

Each element in an array is of the same data type, all elements must be integers, or all elements must be strings of characters, for example. Elements are stored in contiguous memory locations which allows for efficient and direct access to any element in the array using its index.

Consider the following array.

Animal(0)="Bird"
Animal(1)="Cow"
Animal(2)="Frog"
Animal(3)="Cat"
Animal(4)="Dog"

This array contains 5 elements, with indexes 0 through to 4. The data is the animal names, currently Animal(3) contains the data "Cat". The index may itself be a variable, for example if i=2, then Animal(i) is referencing "Frog".

In Chapter 1.3 we look at a range of different data structures in detail, including arrays.

Below is a visualisation of the Animal array:

| Animal(0) | Animal(1) | Animal(2) | Animal(3) | Animal(4) |
|---|---|---|---|---|
| Bird | Cow | Frog | Cat | Dog |

*Fig 1.2.27*

## Recursion

Recursion is a further control-like structure that is particularly useful when problem are composed of small subproblems that are a simpler version of the larger problem. It is not an easy concept to master so looking at a range of examples and trying to solve problems is the best way to become familiar with the technique.

Essentially recursion means that a subprogram (generally in the form of a function) calls itself from within. If this was all that occurred then, as you might imagine, the call within the subprogram when executed initiates a further call to the subprogram which in turn includes a further call to the subprogram, and so on forever. There needs to be a way to end or resolve the subprogram without a recursive call. This is known as the base case, and there may be more than one base case.

To reach the base case in the nested sequence of calls, requires that the input parameter to each subsequent call gets closer to the base case as we go deeper and deeper. Eventually a call occurs that is the base case, and hence can be resolved. Once this occurs, each of the higher level calls are able to resolve, all the way up the chain to the original call.

Let's look at a relatively simple example of recursion, the Fibonacci sequence.

### Fibonacci sequence

Mathematically the Fibonacci sequence is defined as follows.

- First term equals zero.
- Second term equals one.
- Subsequent terms are the sum of the preceding two terms.

The first 15 terms of the Fibonacci sequence is therefore as follows – 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377.

We would like to design an algorithm that returns the nth term of the Fibonacci sequence.
An iterative algorithm could be as follows.

```
BEGIN iFib(n)
      arrfib(1) = 0
      arrfib(2) = 1
      FOR i = 3 TO n
            arrfib(i) = arrfib(i-1) + arrfib(i-2)
      NEXT i
      Return arrfib(n)
END iFib
```

*Fig 1.2.28*

This algorithm does work correctly, and it reflects the above maths definition well, apart from the use of the FOR loop. A recursive algorithm performs the same task without requiring a loop.

A recursive algorithm could be written as follows.

```
BEGIN rFib(n)
      IF n=1 THEN
            result = 0
      ELSEIF n=2 THEN
            result = 1
      ELSE
            result = rFib(n - 1) + rFib(n - 2)
      ENDIF
      RETURN result
END rFib
```

*Fig 1.2.29*

Note that within the recursive rFib algorithm there are two calls to the rFib algorithm itself, these are examples of recursive calls. No such recursive calls are present in the iterative iFib algorithm.

Notice that the iFib example uses an array to store the Fibonacci sequence values as they are generated. In the iterative version, it is each of the recursive calls that stores the prior values in the sequence.

A desk check of the iterative iFib algorithm to find the 6th element of the sequence is as follows.

| arrFib(1) | arrFib(2) | i | arrFib(i) |
|---|---|---|---|
| 0 | 1 | | |
| | | 3 | 1 |
| | | 4 | 2 |
| | | 5 | 3 |
| | | 6 | 5 |

*Fig 1.2.30*

A call tree describing the operation of the recursive rFib algorithm is as follows. The black arrows indicate each call as it is made from the call above, the red arrows indicate the value returned by the call below.

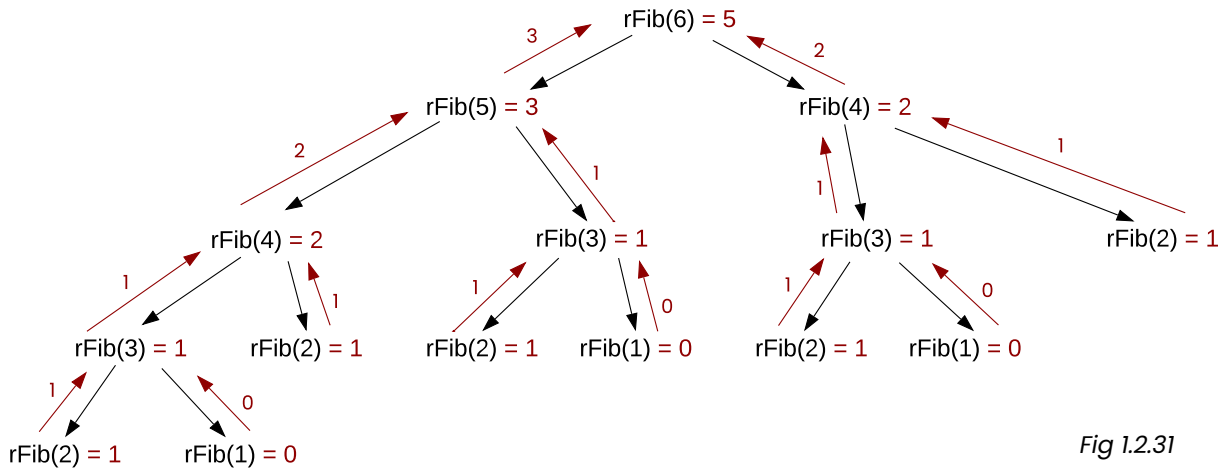



*Fig 1.2.31*

In Fig 2.1.31 above, execution follows the black arrows down to the lower leaves of the tree, which are each examples of base case calls and therefore can be resolved without recursion. As each call is able to complete it passes its return value back up the tree. Finally values for rFib(5) and rFib(4) are returned to the top level or root of the tree. The rFib(6) call can then be completed resulting in the value 3 + 2 = 5 being returned.

## CLASS DISCUSSION

**Compare: Iterative vs recursive**
Compare and contrast the iterative and recursive solutions to the Fibonacci problem above. Some areas to consider include.

- Ease of understanding the logic.
- Number of times the addition of two terms takes place.
- What would happen for large values of n?

**Discuss: Suggestions to improve algorithm**
Can you suggest improvements to either algorithm to improve efficiency?
What changes would be needed to output the first n terms rather than just the nth term?

## EXERCISE 1.2.2 - MULTIPLE CHOICE

1. Breaking a large problem into a series of smaller component problems is known as:

   A. Top-down design
   B. Bottom-up design
   C. Divide and conquer
   D. Backtracking

2. Which design strategy is most likely to use recursion?

   A. Top-down design
   B. Bottom-up design
   C. Divide and conquer
   D. Backtracking

3. Creating smaller self-contained parts and then combining them to solve a larger problem is a good description of which design strategy?

   A. Top-down design
   B. Bottom-up design
   C. Divide and conquer
   D. Backtracking

4. Exploring different paths to find solutions in a systematic manner and rejecting paths that fail to yield a solution, is known as:

   A. Top-down design
   B. Bottom-up design
   C. Divide and conquer
   D. Backtracking

5. How is data passed best between subroutines?

   A. Via parameters
   B. Using global variables
   C. Saving to a file
   D. User inputs

6. A function where the inputs and outputs are known but there is no need to know the detail of processing is called a:

   A. white box
   B. black box
   C. subprogram
   D. module

7. A subroutine includes a call to itself. The call to itself is an example of:

   A. Iteration
   B. Backtracking
   C. Recursion
   D. Modularity

8. The subroutine DoThat includes the line DoThis(x, y). The values of x and y are altered by the DoThis subprogram and yet the value of x and y have not changed for subsequent code in the DoThat subprogram. Of what is ths an example?

   A. Recursion
   B. By reference parameter passing
   C. By value parameter passing
   D. Returning a value

9. Which modelling tool best shows the top-down design, parameters and order of processing?

   A. Data flow diagram
   B. Hierarchy chart
   C. Flowchart
   D. Structure chart

10. Why is a base case needed when using recursion?

    A. To enable the first recursive call to be resolved once all other recursive calls have been made.
    B. To ensure there is an option that does not result in a recursive call, thus enabling the sequence of recursive calls to be resolved.
    C. The base case ensures the sequence can begin to be generated recursively, without it the processing would continue infinitely.
    D. The base case is the simplest example of the problem and therefore it is required so future developers are able to efficiently grasp the logic of the design.

## EXERCISE 1.2.2 – SHORT ANSWER

11. Modular design is a feature of all well written software. Outline advantages of a modular design approach.

12. Distinguish between a recursive call and a non-recursive call. Use examples to illustrate the difference.

13. Create a top-down design using a hierarchy chart to describe the subprocesses required to drive a car from point A to point B. Include starting the car, negotiating a roundabout, a set of traffic lights, maintaining speed, slowing, accelerating, turning, etc.

14. The following algorithm describes the logic required to add a new book to a library catalogue based on its ISBN. The ISBN is the International Standard Book Number, a globally unique number to identify books.

```
BEGIN AddNewBook( ISBN )
     IF CheckDigitOK( ISBN ) THEN
          NewBook = RetrieveBookInfo( ISBN )
          IF NewBook.Title is NOT Null THEN
               SaveBook( NewBook )
               Display NewBook.Title & " by " & NewBook.Author " SAVED"
               RETURN True
          ELSE
               Display "No book with that ISBN could be found"
               RETURN False
          ENDIF
     ELSE
          Display "Invalid ISBN"
          RETURN False
     ENDIF
END AddNewBook
```

Draw a structure chart to model the AddNewBook algorithm above.

15. Summing the elements in an array of numbers is a common problem. It can be solved using iteration, but it can also be solved using recursion. The sum of an array can be defined recursively as the first element plus the sum of the remaining elements.
Write an iterative algorithm and a recursive algorithm to return the sum of the elements in an array of integers.

## Time Complexity (and Big O notation)

Big O notation is used to compare algorithms. It describes the performance or time complexity of algorithms compared to the number of data items being processed. The mathematical formulation of Big O for individual algorithms is beyond the Software Engineering HSC course; however a general understanding of the most common classifications will assist when comparing the performance of standard and custom algorithms. Big O describes the worst case behaviour of an algorithm, such as searching for an item that isn't present in the data.

Common Big O classifications from fastest to slowest are briefly described below followed by a graph and table to illustrate the relative differences.

- **O(1)** – The processing time is constant regardless of the number of data items. For example, displaying the first data item in an array is O(1). It takes the same time regardless of the size of the array.
- **O(log n)** – Processing time increases slower as n increases in size. For instance, it may take 1ms (millisecond) to process 10 data items, 2ms to process 100 items and 3ms to process 1000 data items. The binary search is an example of an O(log n) algorithm.
- **O(n)** – Processing time is proportional to the number of data items. For example, if the number of data items is doubled then the time also doubles. This is very common as any algorithm which performs processing on each data item will be O(n). Linear searches are an example of an O(n) algorithm.
- **O(n log n)** – Processing of each data item is an O(log n) process. As there are n data items then the time complexity is O(n log n). Efficient sorting algorithms are O(n log n), for example Quick sort and Merge sort.
- **O($n^2$)** – Processing of each data item potentially requires examining all other data items. This means time increases rapidly as the number of data items n increases.
- **O($2^n$)** – Processing time increases exponentially as the number of data items n increases. O(2n) algorithms are of little use unless you are certain n will always remain small.
- **O(n!)** – Factorial time complexity is the worst case. The running time grows enormously as n increases hence such algorithms should be avoided.

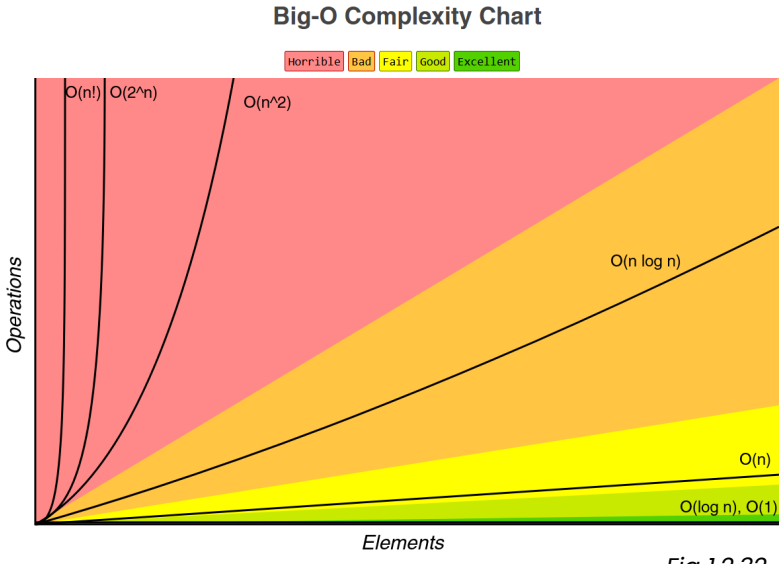



*Fig 1.2.32*

The graph in Fig 1.2.32 shows the scale of the differences between each classification and these enormous differences become increasingly more significant as n increases. Many algorithms used on a daily basis process many millions of data items, such as when processing graphics, sound and video files. For millions of data items the difference in time between O(log n), O(n) and O(n log n) becomes highly significant. The table in Fig 1.2.33 illustrates the magnitude of the differences in processing time as n increases by a factor of ten. For instance, if you can reduce an O(n) algorithm to an O(log n) algorithm then when processing 1 million data items you reduce the time taken from 1 million units of time down to just 14 units of processing time - a staggering saving. If the algorithm is processing pixels within a bitmap image on a mobile phone then potentially a few seconds of processing is reduced to a virtual instant at no cost.

| Data Items | Processing Time | | | | |
|---|---|---|---|---|---|
| n | O(1) | O(log n) | O(n) | O(n log n) | $O(n^2)$ |
| 10 | 1 | 2 | 10 | 23 | 100 |
| 100 | 1 | 5 | 100 | 461 | 10000 |
| 1000 | 1 | 7 | 1000 | 6908 | 1000000 |
| 10000 | 1 | 9 | 10000 | 92103 | 100000000 |
| 100000 | 1 | 12 | 100000 | 1151293 | 10000000000 |
| 10000000 | 1 | 14 | 10000000 | 13815511 | 1000000000000 |
| 100000000 | 1 | 16 | 100000000 | 161180957 | 100000000000000 |

*Fig 1.2.33*

## Standard Algorithms

Many problems encountered by software developers use similar techniques as part of their solution. It makes sense to develop standard methods of approach rather than continuing to reinvent the wheel. These standard modules can be included in a library of routines for future use. In this section, we examine a number of standard algorithms that perform tasks for searching and sorting data and consider their time complexity. As searching and sorting is such a common task, it is included in most high-level languages.

We will examine standard algorithms for each of the following.

- Linear search
- Finding maximum (or minimum)
- Binary search
- Permutation sort
- Selection sort
- Merge sort

There are two specific algorithmic design strategies specified in the Software Engineering syllabus, "divide and conquer" and "backtracking". Merge sort uses a divide and conquer strategy, so to conclude we will examine a classic backtracking puzzle known as the "N-Queens problem".

## Linear Search

The word linear means straight line. A linear search involves examining items one at a time beginning at the start of the list and continuing to the end of the list. Linear searches do not require the data to be in any particular order.

A list containing 100,000 items will require between 1 and 100,000 comparisons to find a specific item if it exists within the list. For situations where items are not found or we wish to determine the number of times an item exists then all items in the list must be examined. As a consequence, the time taken for linear searches to complete is directly proportional to the number of elements in the list.

For instance, searching a list of 2 million items will take double the time taken to search a list of 1 million items. In Big O notation a linear search is an O(n) algorithm. n is the number of items in the list and O(n) essentially means there is a straight line which converts the number of items n to the processing time required O(n).

Each item in the array is examined in turn. The word "Found is displayed each time the required item is encountered.

```
BEGIN LinearSearch
      Set Count to 0
      Get itemToFind
      WHILE more items in item array
            IF item(Count) = itemToFind
                  Display "Found"
            ENDIF
            Add 1 to count
      ENDWHILE
END LinearSearch
```

*Fig 1.2.34*

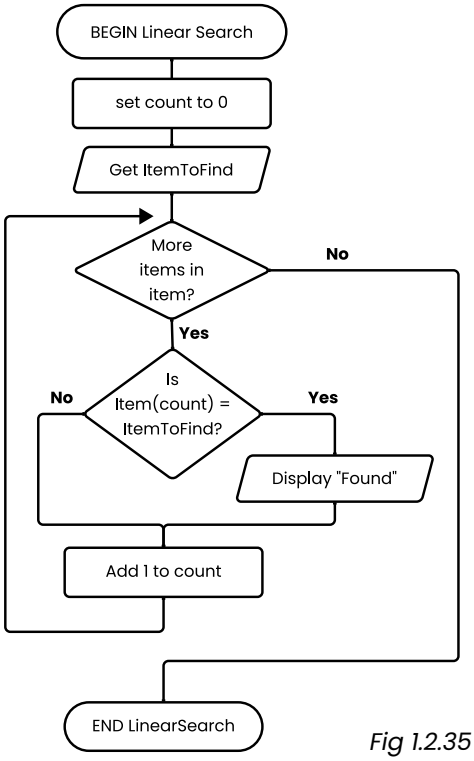



*Fig 1.2.35*

CLASS DISCUSSION

**Discuss: Linear searches**
Explain how each of the linear search algorithms below differ from the algorithm shown in Fig 1.2.34

**Linear search 1:**

```
BEGIN LinearSearch(itemToFind, item)
        Set Count to 0
        Found = False
        WHILE more items in item AND Not Found
                Found = (item(Count) = itemToFind)
                Add 1 to count
        ENDWHILE
        Return Found
END LinearSearch
```

**Linear search 2:**

```
BEGIN LinearSearch(itemToFind, item)
        Set Count to 0
        NumFound = 0
        WHILE more items in item
                IF(item(Count) = itemToFind) THEN
                        Add 1 to NumFound
                ENDIF
                Add 1 to Count
        ENDWHILE
        Return NumFound
END LinearSearch
```

## Finding Maximum and Minimum Values

Finding maximum and minimum values in an unsorted list of items is essentially performed using a linear search where a record is maintained of the largest or smallest value as the list is traversed. For sorted lists, finding maximum and minimum values is trivial; they will be either the last or the first values in the list.

The first item in the array is initially set as the maximum (or minimum) value. Each item in the array is compared in turn with the maximum (or minimum) value. If an item is found to be larger (or smaller) then it becomes the maximum (or minimum) value. This process continues until the end of the list is reached.

**Maximum search:**

```
BEGIN FindMaximum
        Set count to 1
        Set maximum to 0
        WHILE more items in item array
                IF item(count) > item(maximum) THEN
                        Set maximum to count
                ENDIF
                Add 1 to count
        ENDWHILE
        Display item(maximum)
END FindMaximum
```

*Fig 1.2.36*

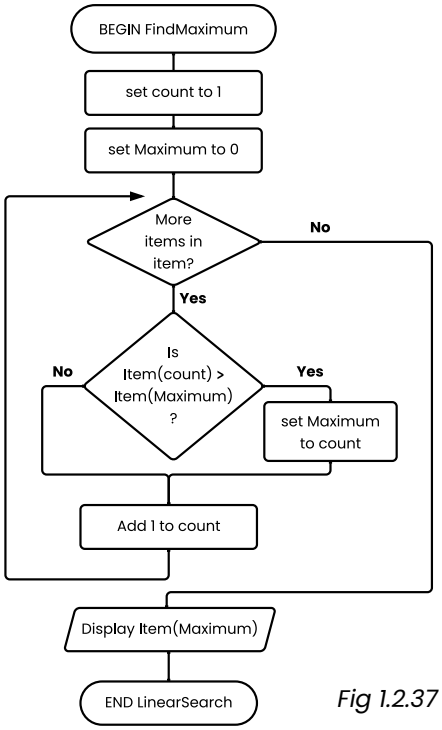



*Fig 1.2.37*

CLASS DISCUSSION

**Discuss: Maximum/Minimum searches**
In the above algorithms (Fig 1.2.36) is the Item array indexed from 0 or from 1? Perform a desk check of the algorithms to confirm your answer.

**Discuss: Minimum search**
Using a flowchart, write an algorithm to find the minimum value by searching from the end of the list to the beginning of the list.

## Binary Search

The word binary means two. A binary search involves splitting the list into two parts each time a comparison is made. As the list must have been previously sorted, one half of the list can be discarded after each comparison is made. Eventually the required item will be found. Binary search is a classic example of a divide and conquer strategy.

A list of items containing 100,000 items will after 1 comparison be reduced to 50,000 items, after two comparisons to 25,000 items, 3 comparisons 12,500 items, and so on. For 100,000 items it will take a maximum of only 17 comparisons to locate a specific item in the list. As $2^{16}$ = 65,536 and $2^{17}$ = 131,072, and there are 100,000 items, and 100,000 lies between $2^{16}$ and $2^{17}$ , the maximum number of comparisons is 17.

In general, a list containing X items will take lnX/ln2 (rounded up to the nearest integer) comparisons to find any particular item. Binary search is a O(log n) algorithm and is therefore significantly more efficient than a linear search algorithm. However for a binary search the data must be sorted, so if this is not already true or is not practical then a linear search may be a better solution.

The list of data items must be sorted. The middle item in the list is compared to the required item. If the required item lies in the first half of the list then the second half is discarded. If the required item lies in the second half of the list then the first half is discarded. This process is repeated with the remaining list of items. Eventually the required item will be found or the list of possible items will be empty.

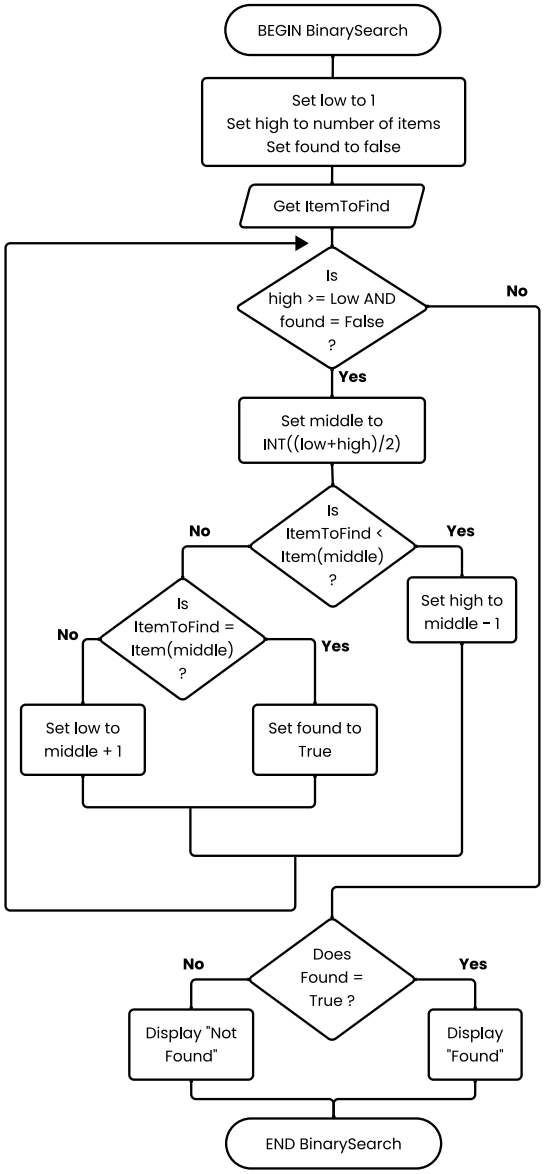



*Fig 1.2.38*

**Binary search pseudocode:**

```
BEGIN BinarySearch
        Set Low to 1
        Set High to number of items in list
        Set Found to False
        Get ItemToFind
        WHILE High >= Low AND Found = False
                Set Middle to INT((Low + High)/2)
                IF ItemToFind < Item(Middle) THEN
                        Set High to Middle - 1
                ELSEIF ItemToFind = Item(Middle) THEN
                        Set Found to True
                ELSE
                        Set Low to Middle + 1
                ENDIF
        ENDWHILE
        IF Found = True THEN
                Display "Found"
        ELSE
                Display "Not found"
        ENDIF
END BinarySearch
```

*Fig 1.2.39*

A list of animals has been sorted into alphabetical order. The list is stored in an array called Animals. A binary search is required to find if a particular animal already exists within the array Animals. Let us consider the following sample Animals array.

Using the binary search algorithm, with the Items array replaced by the Animals array, we would first set Low to 1 and High to 10. Found is also set to False.

Animals:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| Ant | Beetle | Cow | Dog | Frog | Goose | Hen | Moose | Rat | Yack |
| Low | | | | | | | | | High |

We now get an ItemToFind, say Cow. As High is greater than Low and Found is False we progress into the body of the loop and set Middle to INT((1 + 10)/2), which is 5.

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| Ant | Beetle | Cow | Dog | Frog | Goose | Hen | Moose | Rat | Yack |
| Low | | | | Middle | | | | | High |

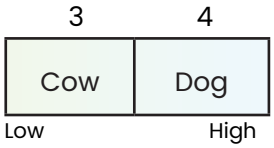
We now compare the required item Cow with the data item associated with 5, namely Frog. As Cow is less than Frog, in an alphabetical sense, High is set to 4, the current value of Middle - 1. In essence, we have discarded the top half of the array.

| 1 | 2 | 3 | 4 |
|---|---|---|---|
| Ant | Beetle | Cow | Dog |
| Low | | | High |

We now repeat the process. Middle is set to INT((1 + 4)/2), which is 2.

| 1 | 2 | 3 | 4 |
|---|---|---|---|
| Ant | Beetle | Cow | Dog |
| Low | Middle | | High |

Cow is compared to item Animals(2). It is not less than Beetle and it is not equal to Beetle, so Low is set to 3, the current value of Middle + 1. The lower half of the array is discarded.




The process repeats again. Middle is set to INT((3 + 4)/2), which is 3. We compare the required item with Animals(3) and find we have a match. Found is set to True and we then drop out of the loop. As Found is True the message Found is displayed.

## CLASS DISCUSSION

**Activity: Binary search - Animals**
Choose an animal that is not in the above array. Work through the binary search to ensure the algorithm operates correctly and the output is Not found.

**Activity: Binary search - Numbers**
Write down a list of 30 numbers in order from lowest to highest. Select one of the numbers and perform a desk check of the binary search algorithm.

## EXERCISE 1.2.3 - MULTIPLE CHOICE

1. A search that involves comparing each item in turn with the required item is called a:

    a. Binary search.
    b. Descending search.
    c. quadratic search.
    d. linear search.

2. A sorted list is one of the requirements of a:

    a. binary search.
    b. descending search.
    c. quadratic search.
    d. linear search.

3. Finding the maximum data item in a list is relevant for:

    a. numeric data.
    b. string or text data.
    c. a sorted list.
    d. both A and B.

4. A search of a data list containing 20,000 items takes on average 10,000 comparisons to find a particular item. The type of search being used is probably a:

    a. binary search.
    b. minimum search.
    c. maximum search.
    d. linear search.

5. Functions and procedures are used toimplement algorithms. Which of the following statements is true?

    a. Procedures always return one value; functions can return any number of values.
    b. Functions always return one value; procedures can return any number of values.
    c. Functions are only able to process numeric data.
    d. All functions must include at least one input parameter.

6. The process of progressively breaking a problem down into its component parts is known as:

    a. algorithm development.
    b. top-down design.
    c. system modelling.
    d. stepwise refinement.

7. A library of routines is:

    a. used to store standard algorithms for reuse.
    b. used to store standard modules of source code for reuse.
    c. of use to end users who wish to modify the original product.
    d. a common way of storing the top-down design of a program.

8. An algorithm description language:

    a. provides excellent user documentation.
    b. is either a flowchart or pseudocode.
    c. provides a method of expressing algorithms.
    d. is a type of programming language.

9. Any particular item is always found in a list of 1000 items within 10 comparisons. What can be said about the 1000 items?

    a. There must be only 10 unique items in the list.
    b. The list of items must be sorted.
    c. Most of the list must be empty.
    d. There is insufficient information to answer this question.

10. The largest value in an array of 10,000 items is found to be in position 234. Which of the following is True?

    a. The list must be sorted.
    b. Larger items may exist in positions 235 to 10,000.
    c. The list must be unsorted.
    d. The list only contains numeric data.

## EXERCISE 1.2.3 - SHORT ANSWER

11. Often, with numeric decimal values, a problem will involve finding a value within a specific range. Design a linear search algorithm that will search for and display all the values in a list between a lower and upper value.

12. Create an algorithm to find and output both the maximum and the minimum values stored in an array of unsorted data items.

13. Perform a desk check of both the linear search algorithm and the binary search algorithm using the following test data: The Item array contains the 6 elements CPU, Keyboard, Monitor, Mouse, Printer and Scanner indexed from 1 to 6. The input to the algorithms is Printer.

14. An array contains a sorted list of 2000 people's surnames. Design an algorithm using a binary search strategy that will find the number of occurrences of a particular surname.

15. Your English teacher wishes you to develop a program to tally marks for them. The marks are to be tallied into deciles. For example, a mark of 72 is in the 8th decile, 36 is in the 4 th decile and a mark of 6 is in the 1 st decile. Design an algorithm to accomplish this task. Assume the marks are stored in an unsorted array called Marks, which is indexed from 0 to 99.

## Permutation Sort

This first example of a sorting algorithm would never be used in practice. We include it as an example of what is possible, but not wise!

Imagine a random set of 5 different playing cards that we wish to arrange into ascending alphabetical order. For a permutation sort we essentially throw the cards on the table and then randomly pick them up in any order. We then look at the cards in our hand to see if they are in sorted order. If they are not in order, then we toss them on the table again, pick them up in random order again and see if they are correct this time. The process repeats until we "fluke" picking up the cards in sorted order.

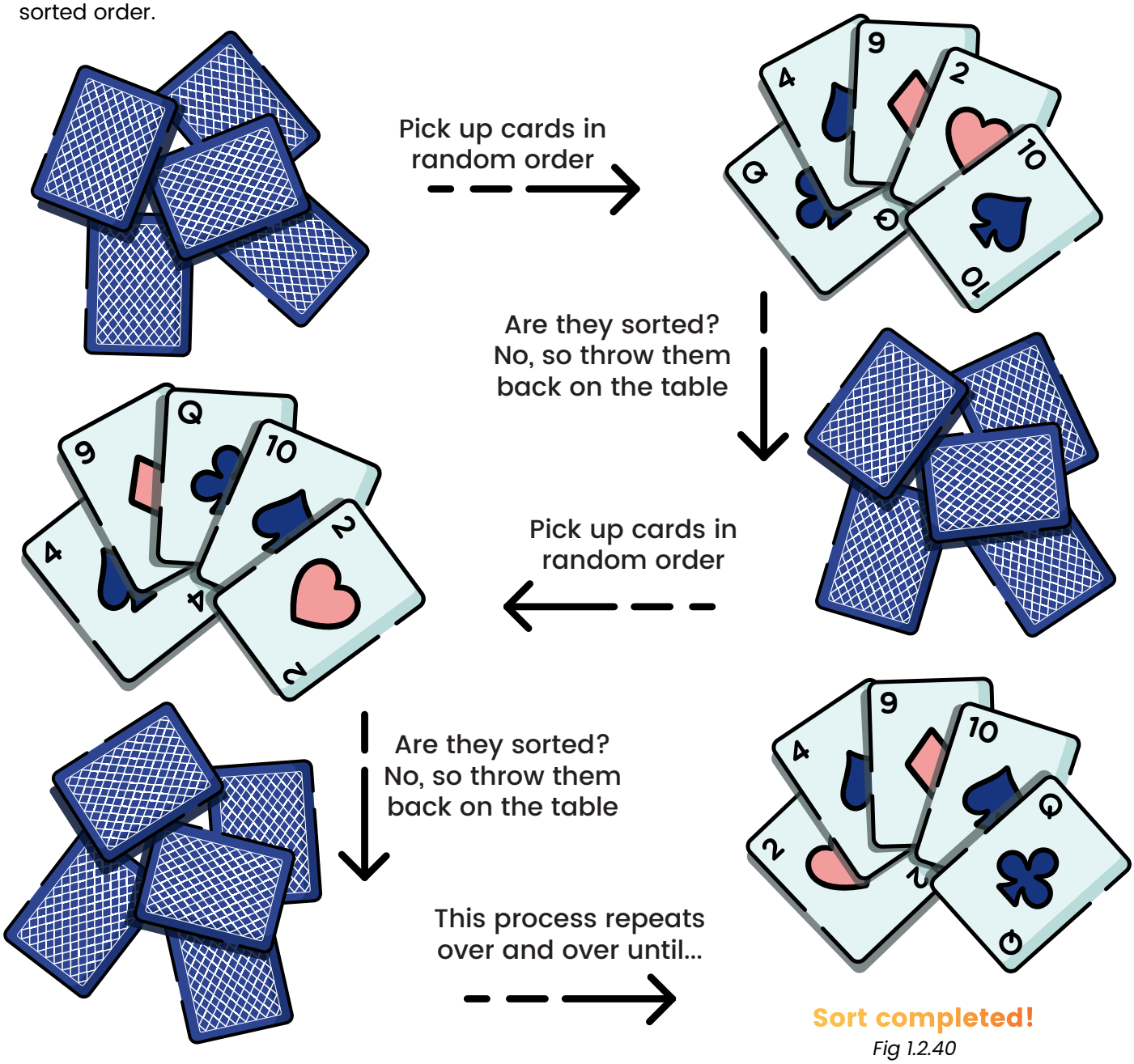



*Fig 1.2.40*

**Permutation Sort: The simplified concept**

```
BEGIN PermutationSort
       WHILE NOT sorted
              Generate a permutation of the data
              Check if permutation is sorted
       ENDWHILE
END PermutationSort
```

*Fig 1.2.41*

How do we generate each permutation? In our card example there are 5 cards. This means there are five possible random cards available for the first pickup. Given this first card is in position 1, there remains 4 cards we can randomly choose for the second pickup, 3 cards for the third pickup, 2 for the fourth, and the final 5th card must go in the last position.

There is a total of 5 X 4 X 3 X 2 X 1=120 possible different permutations of 5 cards. 5 factorial or 5! Imagine sorting a pack of 52 playing cards this way let alone a database with millions of records.

In general, if there are n elements to be sorted, there will be n! different permutations. Just one of these unique permutations will be sorted correctly. The probability of generating the correctly sorted permutation is 1 in n!. Therefore, the while loop in the above algorithm will need to iterate from 1 to n! times. Worst case is n!, so we're already looking at a time complexity of $O(n!)$ – a disaster!

To be rigorous we should also consider the time taken to check if the data permutation is sorted. This requires checking each element in turn is less than the next element. This operation requires stepping through all elements in turn, therefore the time complexity is $O(n)$.

We therefore have up to n! iterations times n. Total time complexity is $O(n.n!)$. The n is not significant compared to the n!, so the time complexity for our permutation sort remains a shockingly inefficient $O(n!)$.

## CLASS DISCUSSION

**Activity: Permutation sort**
Consider a permutation sort of 1 hundred, 1 thousand, 10 thousand, 100 thousand, and 1 million items. Is it likely these sorts will ever finish?

**Discuss: Permutation time complexity**
We neglected to consider the time taken to actually generate a random permutation of the data. Discuss the time complexity of this operation, whilst discussing the design of an algorithm to accomplish this task.

**Activity: Permutation sort algorithm**
Design an algorithm to check if a given permutation is in sorted order. Assume each element is held in an array.

## Selection Sort

Selection sorts essentially are a repetition of a linear search to find the smallest (or largest) item in a list. Each time the search is complete, the item found is moved to the end of the sorted list. This type of sort is rather more intuitive for humans as it is a strategy often employed when sorting things by hand. For example, when playing card games we often arrange the cards in our hand using this selection sort method.

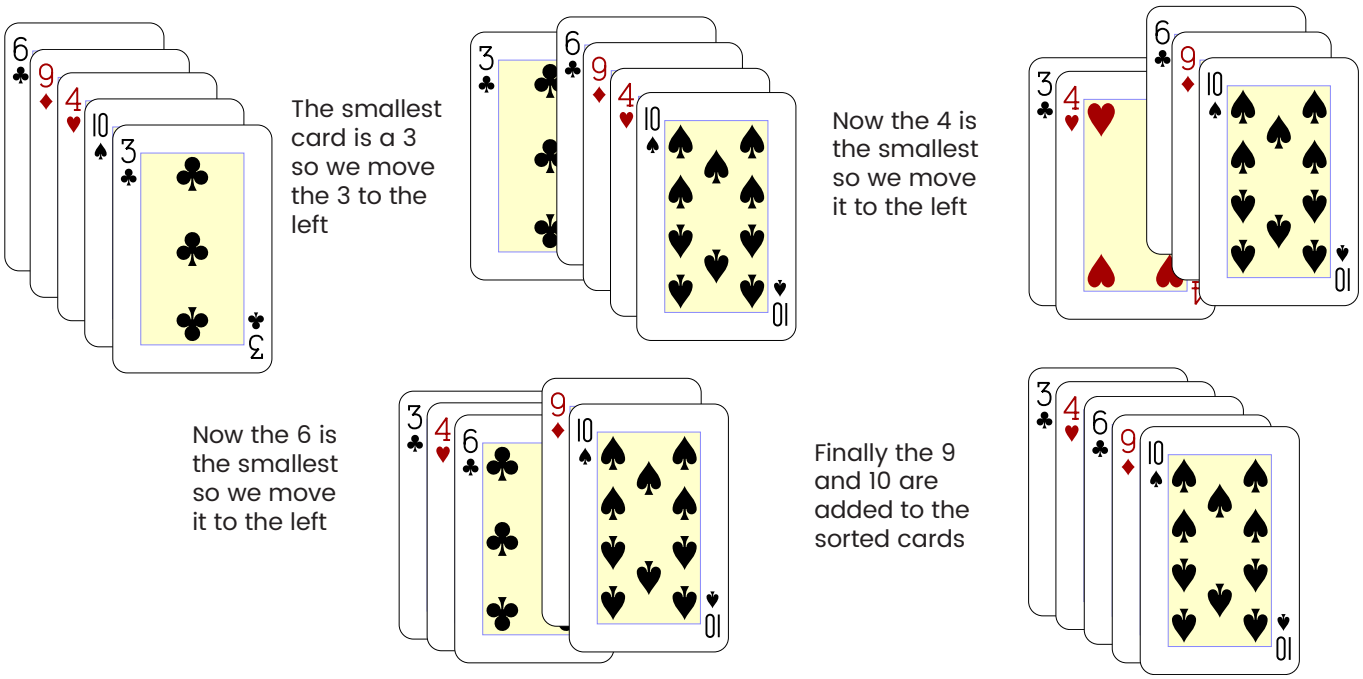



*Fig 1.2.42*

A sorted and an unsorted list are maintained. The smallest item in the unsorted list is swapped with the first item in the unsorted list. The sorted list is increased in length to include the new item. This process is repeated while there are items remaining in the unsorted list.

**Selection Sort: The simplified concept**

```
BEGIN SelectionSort
        WHILE unsorted list has items
                Find smallest item
                swap with first item in list
                increase sorted list size by 1
        ENDWHILE
END SelectionSort
```

*Fig 1.2.43*

### CLASS DISCUSSION

**Activity: Selection sort**
Write about 20 different words on separate slips of paper. Use a selection sort strategy to sort the words into alphabetical order.

**Discuss: Selection sort time complexity**
Selection sorts in Big O are $O(n^2)$. Based on the practical activity above, can you explain why this is the case?

The list is examined one item at a time for the smallest item.
This item is swapped with the first item in the list.
This first item is now sorted and becomes the sorted list.
The remaining unsorted list is examined for the smallest item.
This item is swapped with the first item in the unsorted list.
The length of the sorted list is increased by one. There are now two items in the sorted list. This process is repeated while there are still items remaining in the unsorted list.

**Selection Sort: The enhanced concept**

```
BEGIN SelectionSort
    Pass = 1
    WHILE Pass < Number of items
        Count = Pass + 1
        Minimum = Pass
        WHILE Count <= Number of Items
                IF Item(Count) < Item(Minimum) THEN
                        Minimum = Count
                ENDIF
                Count = Count + 1
        ENDWHILE
        Swap Item(Minimum) with Item(Pass)
        Pass = Pass + 1
    ENDWHILE
END SelectionSort
```

*Fig 1.2.44*

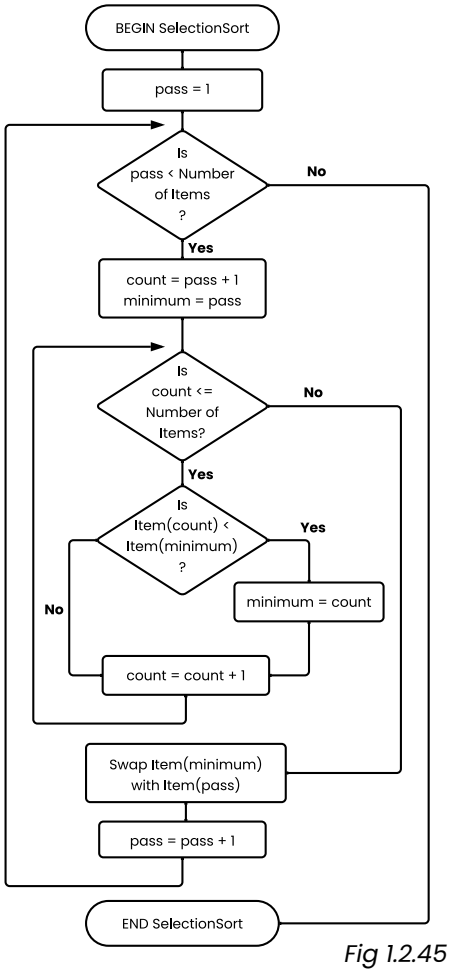



*Fig 1.2.45*

The array of animals below is to be sorted using the selection sort described in the algorithm in Fig 1.2.44 and Fig 1.2.45.
Firstly, we search for the smallest item in the list, namely Ant:

Animals:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| Goose | Yak | Ant | Dog | Moose | Cow | Hen | Rat | Frog | Beetle |

Ant is swapped with the first item, Goose. The new first item, Ant, now forms the sorted part of the array. The unsorted list is searched for the smallest item, namely Beetle:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| ***Ant*** | Yak | Goose | Dog | Moose | Cow | Hen | Rat | Frog | Beetle |

Beetle is swapped with the first item in the unsorted list, namely Yak. The unsorted list is again scanned for the smallest item, and Cow is found:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| ***Ant*** | ***Beetle*** | Goose | Dog | Moose | Cow | Hen | Rat | Frog | Yak |

Cow and Goose are swapped. The smallest item is now Dog:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| ***Ant*** | ***Beetle*** | ***Cow*** | Dog | Moose | Goose | Hen | Rat | Frog | Yak |

Dog is swapped with itself. The smallest item is now Frog:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| ***Ant*** | ***Beetle*** | ***Cow*** | ***Dog*** | Moose | Goose | Hen | Rat | Frog | Yak |

Frog and Moose are swapped. Goose is now the smallest item:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| ***Ant*** | ***Beetle*** | ***Cow*** | ***Dog*** | ***Frog*** | Goose | Hen | Rat | Moose | Yak |

Goose is swapped with itself. Hen is now the smallest item:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| ***Ant*** | ***Beetle*** | ***Cow*** | ***Dog*** | ***Frog*** | ***Goose*** | Hen | Rat | Moose | Yak |

Hen is swapped with itself. Moose is now the smallest item:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| ***Ant*** | ***Beetle*** | ***Cow*** | ***Dog*** | ***Frog*** | ***Goose*** | ***Hen*** | Rat | Moose | Yak |

Moose is swapped with Rat. Rat is now the smallest item:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| ***Ant*** | ***Beetle*** | ***Cow*** | ***Dog*** | ***Frog*** | ***Goose*** | ***Hen*** | ***Moose*** | Rat | Yak |

Rat is swapped with itself. As only one item remains, the list is now sorted.

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| ***Ant*** | ***Beetle*** | ***Cow*** | ***Dog*** | ***Frog*** | ***Goose*** | ***Hen*** | ***Moose*** | ***Rat*** | ***Yak*** |

CLASS DISCUSSION

**Discuss: Selection sort variables**
Examine the selection sort algorithm and describe how the variables Pass and Count combine to change the length of the sorted and unsorted sections of the array Item.

**Activity: Selection sort algorithm**
Perform a selection sort showing the state of the array after a new item has been placed into the sorted part of the array, using the following data:

34, 44, 23, 35, 34, 55, 42, 12, 52, 16, 16, 17, 56, 78, 26, 23, 41, 29, 30, 42.

This time perform the sort from right to left. That is, search for the largest item each time and swap it with the right-hand item in the unsorted part of the array.

## Merge Sort

Merge Sort is a divide-and-conquer sorting algorithm that was designed for efficiency and guaranteed performance. It follows the principle of breaking down a problem into smaller sub-problems, solving them individually, and then combining their solutions to achieve the final result.

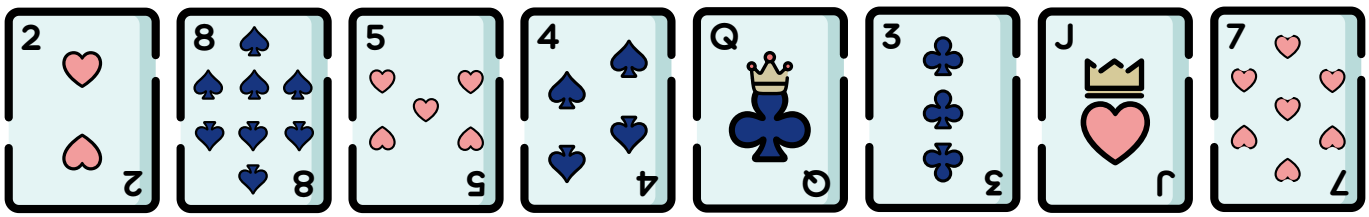
The general idea of Merge Sort can be summarized in the following steps:

- **Divide:** The array to be sorted is recursively divided into two halves until each sub-array contains only one element. This is the base case of the recursion.
- **Conquer:** The individual elements and sub-arrays are then sorted. For sub-arrays with only one element, the sorting is trivial as a single element is always sorted.
- **Combine (Merge):** The sorted sub-arrays are merged to produce new sorted arrays. This is done by comparing the elements of the sub-arrays and arranging them in order. The merging process is repeated until a single sorted array is obtained.

**Key Term: Divide and Conquer**
A problem-solving strategy that breaks down a complex problem into smaller, more manageable versions of the problem, solving them independently, and combining their solutions to solve the original problem.

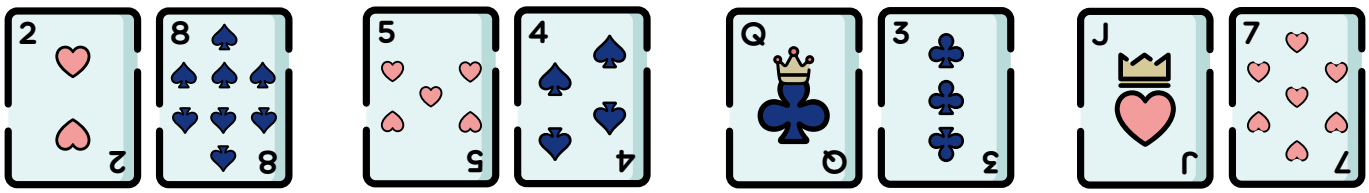
The significant operation in Merge Sort is the merging step. During the merging process, two sorted arrays are combined into a single sorted array. This is achieved by comparing the elements of the two arrays and merging them in the correct order. The merging continues until all sub-arrays are combined into a single, sorted array.

Consider performing a Merge Sort to sort the following eight playing cards.

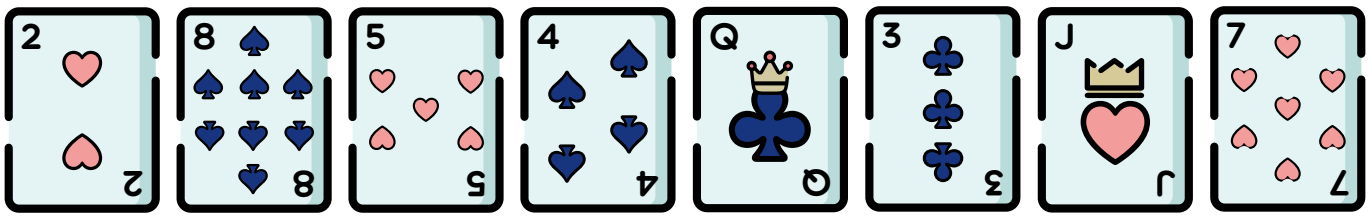


First we divide by splitting the 8 cards into 2 groups of 4 cards each.

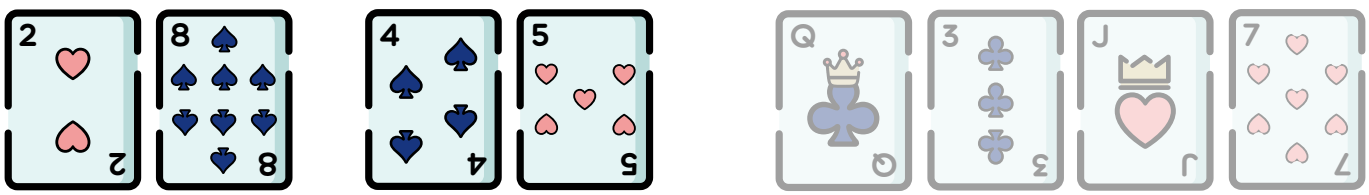
We divide each group of 4 cards into 2 groups of 2 cards.



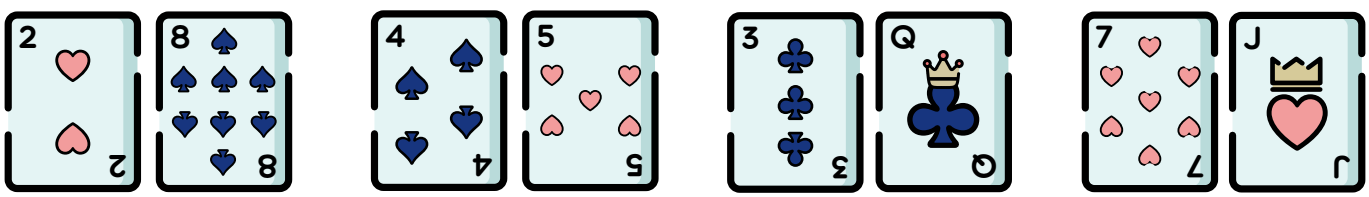
Now we divide each group of 2 cards into 2 single cards.



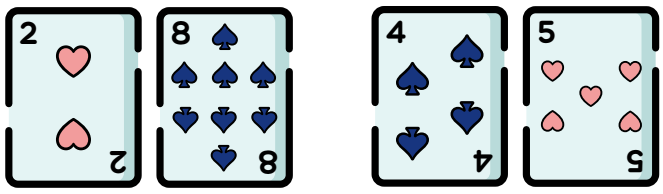
A single card is by definition sorted. We can now begin recombining or merging.
The two is less than eight in the first merge, and four is less than five in the second. The result so far is as follows.



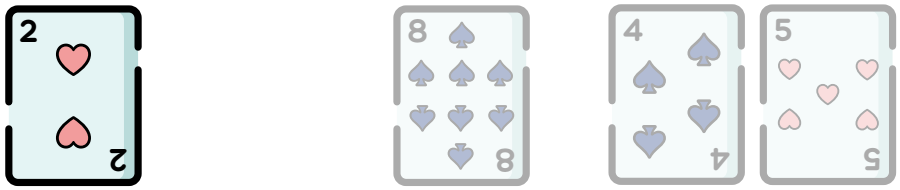
The next merge is the Queen and three, three is lower so comes first. Seven is less than Jack, so comes before Jack.



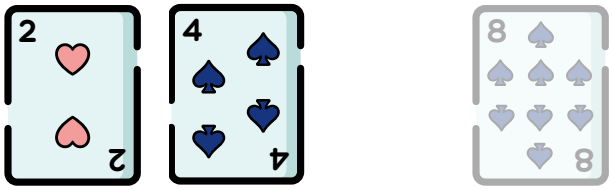
We now begin merging the sorted groups of two into groups of four. First the two, eight is merged with the four, five group.



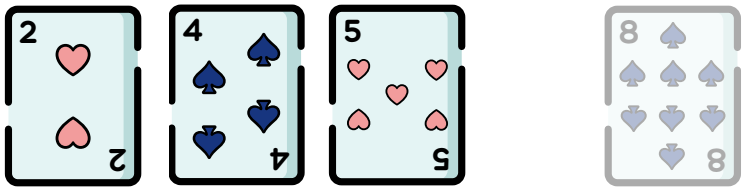
To merge, compare the left card in each group and move the smallest to the new sorted group each time. In this case we take the two as it is less than four.



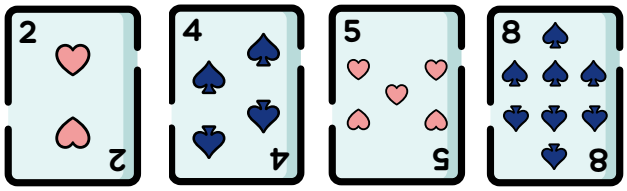
Next compare the eight and the four. Eight is NOT less than four, so we move the four to the sorted group.



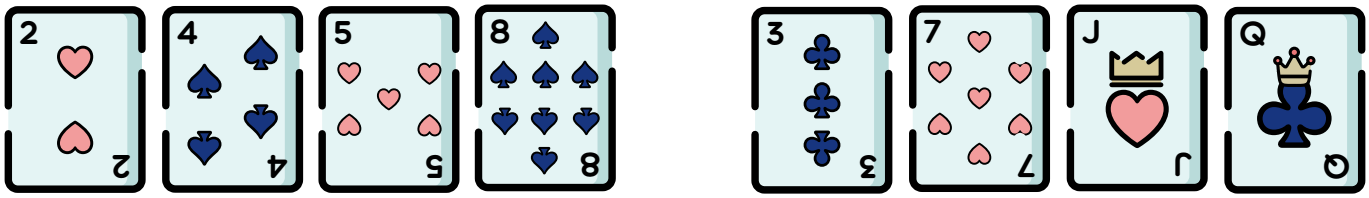
Now compare the eight and the five. Five is smalled so it moves to the sorted group.



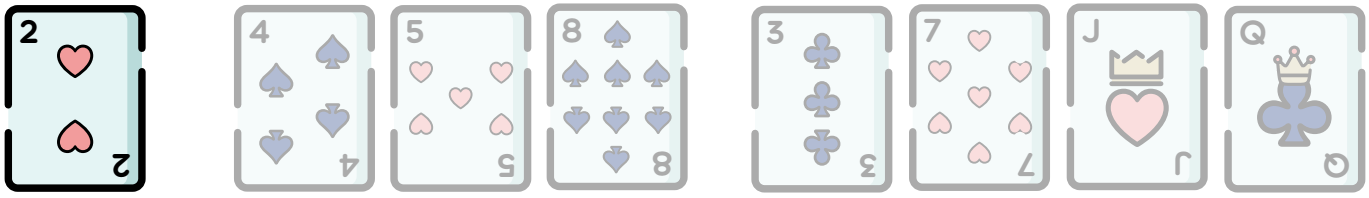
Eight is the only card remaining so must be the largest.



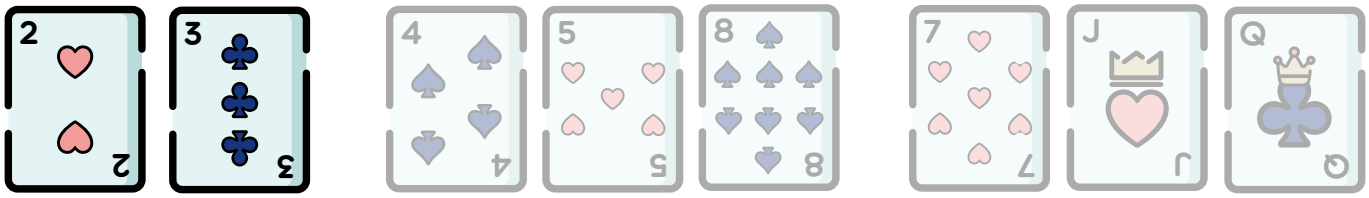
We perform similar comparisons and moves to merge the remaining groups of 2 together.



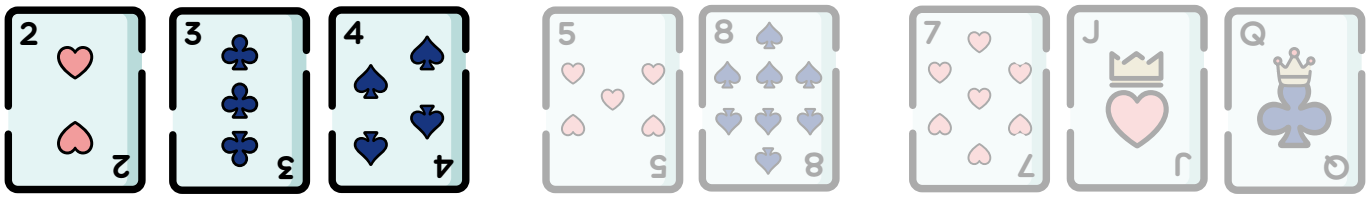
We now have 2 groups of 4 sorted cards each. We need to merge these groups using the same system of comparing the left hand cards in each group and moving the smallest to the new sorted group.
Two is less than three so we move two.



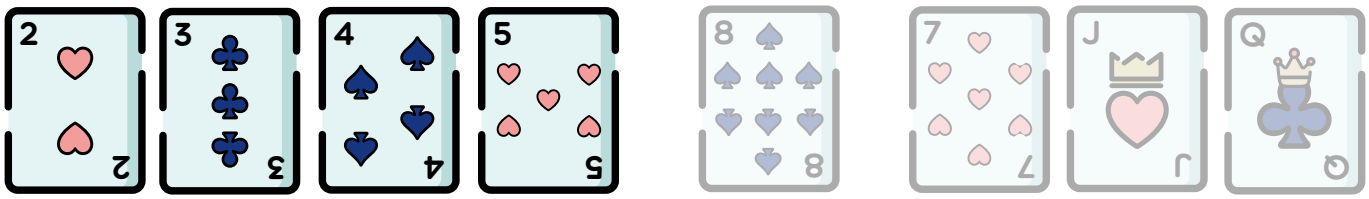
Four is NOT less than three, so we move the three to the sorted group.



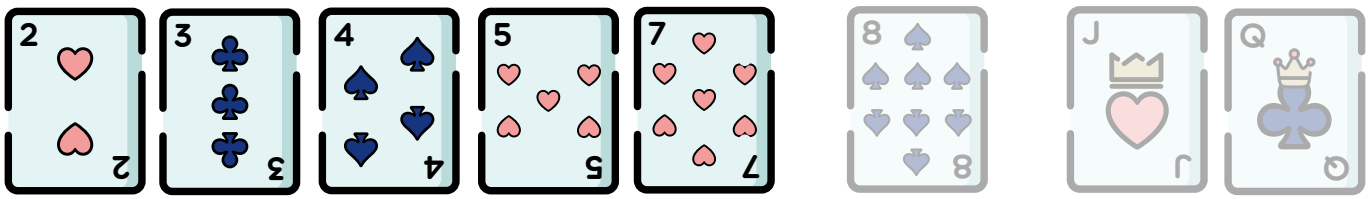
Four is less than seven, so move it to the sorted group.



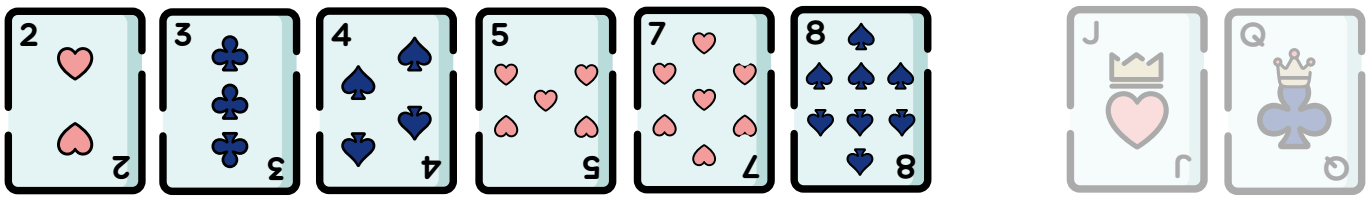
Five is less than seven, so five moves to the sorted group.



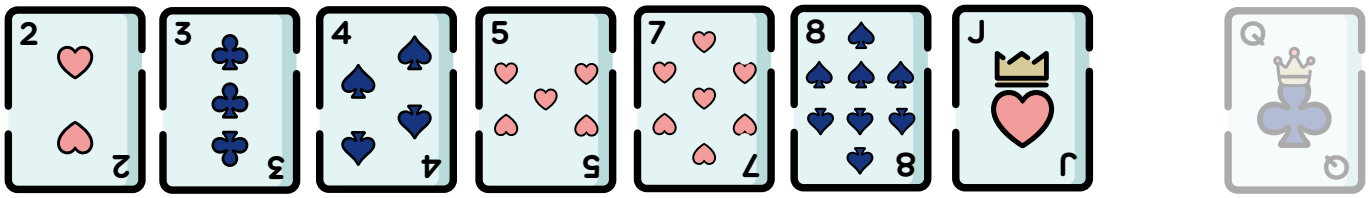
Eight is NOT less than seven, so seven moves to the sorted group.



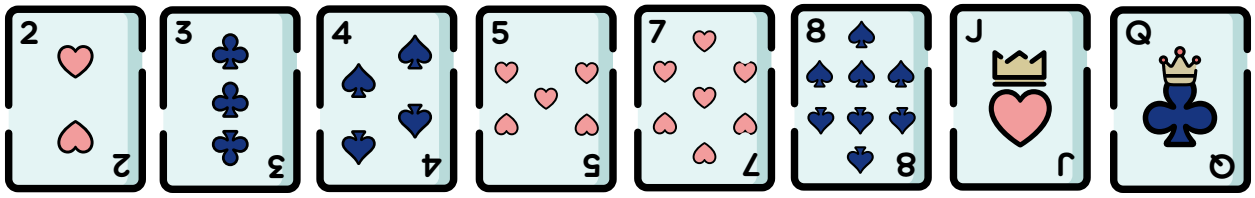
Eight is less than Jack, so eight is moved to the sorted group.



Jack is less than Queen, so moves to the sorted group.



As the Queen is the final card it must be the largest so can be moved to the sorted group.
The cards are now sorted.



**Merge Sort: The simplified concept**

```
BEGIN MergeSort(array)
        IF less than or equal to one element in array THEN
                RETURN array
        ELSE
                Split array into equal sized left and right parts
                MergeSort left part
                MergeSort right part
                Merge left and right parts to form the sorted array
                RETURN sorted array
        ENDIF
END MergeSort(array)
```

*Fig 1.2.46*

Notice the use of two recursive calls to the MergeSort routine within the above MergeSort routine. Whenever the input array provided to the MergeSort routine contains more than one element it is split in two and each of the two parts is sorted using MergeSort. It is only after both MergeSort calls have completed that the array is merged together the call completes. The only other way for a call to complete is when the array contains one (or zero) elements – this is the base case indicating we have finished dividing. The single element MergeSort calls end and then the Merge can take place to begin forming the sorted arrays.

## CLASS DISCUSSION

**Activity: Merge sort with cards**
Shuffle a set of at least 10 playing cards and sort them into order using the above MergeSort algorithm as your guide. Do this multiple times until you have mastered the logic of the MergeSort.

**Merge Sort: The enhanced concept**

```
BEGIN MergeSort(array)
        IF length of array is <=1 THEN
                RETURN array
        ELSE
                middle = length of array divided by 2
                leftArray = elements array(0) to array(middle)
                leftSorted = MergeSort(leftArray)
                rightArray = elements array(middle+1) to array(length of array -1)
                rightSorted = MergeSort(rightArray)
                sortedArray = Merge(leftSorted, rightSorted)
                RETURN sortedArray
        ENDIF
END MergeSort

BEGIN Merge(left, right)
        Set mergedArray to empty array
        WHILE left is not empty AND right is not empty:
                IF left(0) <= right(0)
                        append left(0) to mergedArray
                        remove left(0) from left
                ELSE
                        append right(0) to mergedArray
                        remove right(0) from right
                ENDIF
        ENDWHILE
        RETURN mergedArray
END Merge
```

*Fig 1.2.47*

Let us now think about the work and time complexity of the Merge Sort algorithm.
The divide step splits the *n* data items in the array into two arrays. This is done repeatedly until there is just one item in each array. Each of these splits takes constant time O(1). But how many of these splits have occurred? For *n* items the size of each split array will be *n*/2, then *n*/4, then *n*/8, *n*/16 and so on until there is just a single item in each group.

Now consider the number of levels of splits that are required to split the initial *n* item array into one of the single item arrays. *n* is multiplied repeatedly (say k times) by ½ and the result is 1.

$$n\left(\frac{1}{2}\right)^k = 1 \qquad \frac{1}{2^k} = \frac{1}{n} \qquad n = 2^k \qquad log_2 n = log_2 2^k \qquad log_2 n = k log_2 2 \qquad k = log_2 n$$

There are $\log_2 n$ levels. Note that this is also the same as the total number of splits that have occurred, therefore the time complexity to perform all the splits or divide operations is O(log n) – not a significant amount of time.

On the way back up, to reforming the sorted array, we first merge each single item array to form a 2 item array, then these are merged to form 4 item arrays and so no. This process repeats until we have the fully sorted array containing the original *n* items. Clearly the number of merge operation levels is going to be that same as the number of divide levels, namely $\log_2 n$ but what is the work done at each merge level?

During a merge the left hand element of the two arrays is compared, this occurs repeatedly for all elements. For each compare operation, one element is moved to the larger sorted array. This means there must be *n* compares to move all *n* elements to the sorted array. A total of *n* operations, therefore each level has a time complexity of O(n).

It doesn't matter whether you are merging the one element arrays to form the sorted two element arrays, or merging the final $n/2$ element arrays to form the final sorted n element array, in all cases at each level there will be *n* comparisons required. Therefore, each level of merge operations has a time complexity of O(*n*).

Recall that there are $\log_2 n$ levels, therefore work is done merging array or in Big O notation O(n log n).

The divide operations required O(log n), therefore the total for the entire Merge Sort is O(n log n) + O(log n). O(n log n) quickly makes O(log n) insignificant as n increases, therefore the time complexity for a Merge Sort is simply O(n log n).

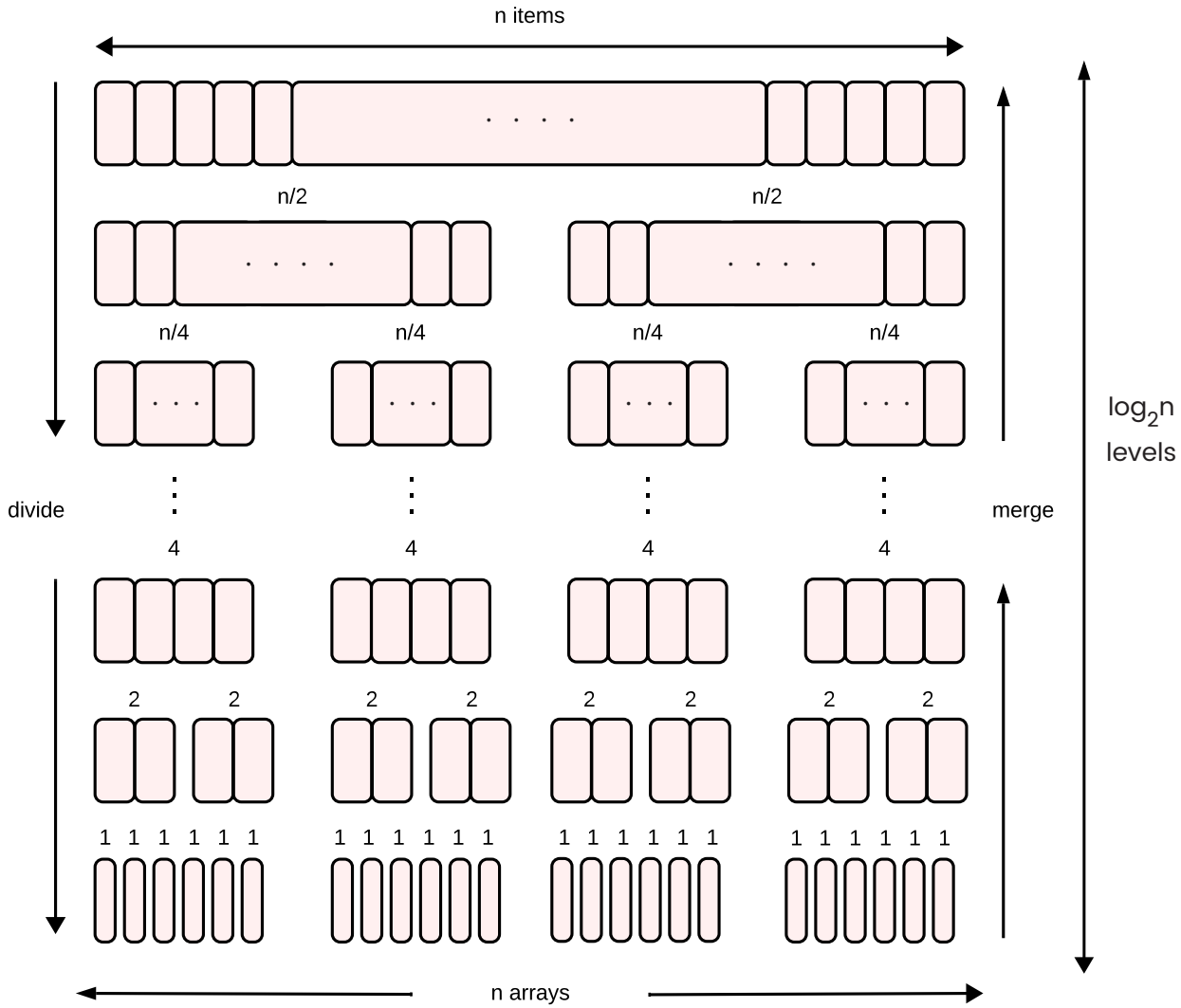



*Fig 1.2.48*

CLASS DISCUSSION

**Activity: Merge sort**
Write about 20 different words on slips of paper. Use a Merge Sort strategy to sort the words into alphabetical order.

**Research: Programming language sorts**
Research the type of sorts used behind the scenes by a number of common high-level languages. Does Merge Sort appear?

## N-Queens Problem

The N-Queens problem is a classic puzzle that has been used to illustrate the backtracking algorithmic strategy to computer science students for decades. The problem itself is challenging, however the aim is not to memorise the solution but rather to grasp the concept of backtracking – keep this in mind as we explore the problem.

The aim is to place N Queens on an N by N chessboard in such an arrangement that no queen threatens any other queen. For those not familiar with chess, the queen is the most valuable piece as she can move any number of squares in a horizontal, vertical or diagonal direction. A real chessboard is an 8 by 8 grid of 64 squares, and so the 8-Queens version of the problem involves placing 8 queens so no queen threatens any of the other 7 queens.

Let us first define a system for identifying each square on the board. Consider the bottom left corner to be the origin, so that square has the address (0,0), moving to the right increases the first dimension and moving up increases the second. For an 8 by 8 board, the bottom right square is (0,7), top left (7,0) and top right (7,7) as shown in Fig 1.2.49. If extended to an N by N board then simply replace 7 with N.

Fig 1.2.49 shows the conflicting squares with a cross and safe squares empty after placing the first queen in square (2,4). Notice that this one queen has already created 25 squares that are under threat and left just 38 spaces as potential locations for the remaining 7 queens.

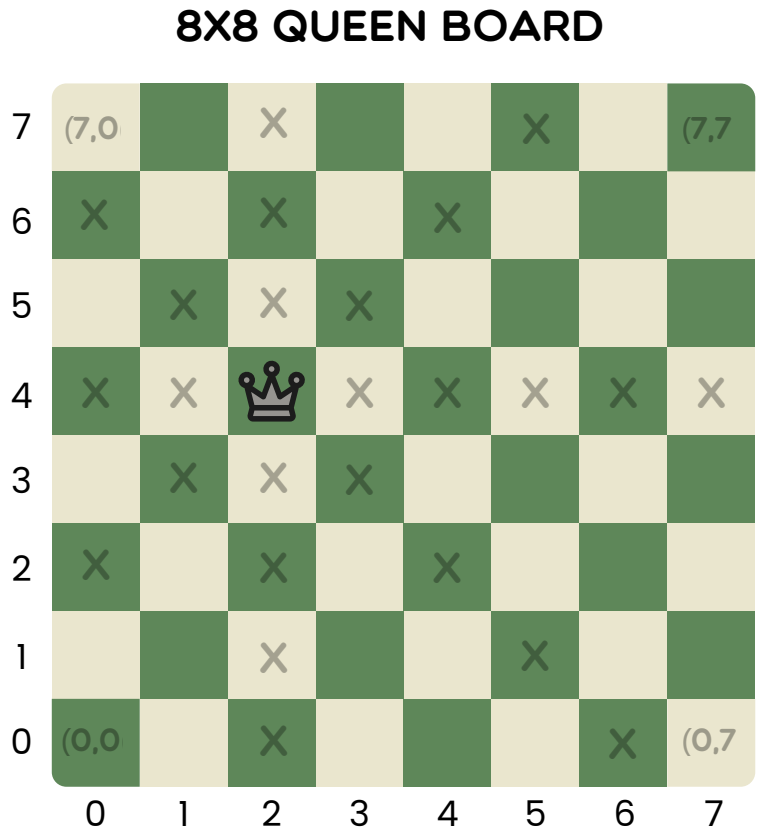



*Fig 1.2.49*

**Activity: N-Queens Problem**
Before reading any further, draw up one or more 8 by 8 grids and see if you can find a solution to the 8-Queens version of the problem. Think carefully about the strategy, or algorithm, you are employing as you search for a solution.

Let's begin by considering smaller versions of the N-Queen problem.

## 1-Queen problem

The smallest occurs when N=1. There is one square (0,0) and one queen so the solution is trivial – place the queen on the (0,0) square. A single solutions exists.

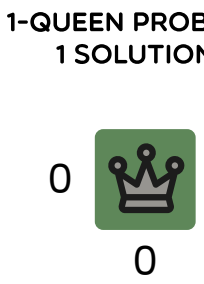


*Fig 1.2.50*

## 2-Queen problem

Consider N=2. This time there are four squares in the grid. Placing the first queen at (0,0) and then consider where to place the second queen. No location is safe as the first queen threatens all other squares. Trying each other square in turn yields no possible solution either, hence there is no solution to the 2-Queens problem.

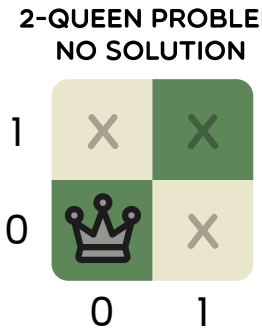


*Fig 1.2.51*

## 3-Queen problem

Consider N=3. This time we have nine squares to place 3 queens. It now not so obvious whether a solution exists.

We need to think of a strategy. Consider all N-Queen problems, as soon as a queen is placed in a column then no other queen can occupy that column so we need only consider remaining columns. So let us place our first queen at (0,0), look for solutions, then move the first queen up the column to (1,0) and explore solutions, and so on.

Initially we place a queen Q1 in (0,0) as shown in Fig 1.2.52. We need not consider column 0 any further as it contains Q1. So we look at column 1 and ask, can a queen go in square (1,0)? No, as that is the same row as Q1 is located. Can a queen go in square (1,1)? No, as this is threatened diagonally by Q1. Moving up column 1 again we ask, can a queen go in square (1,2)? Yes, it can, so we place Q2 into square (1,2) as shown in Fig 1.2.52.

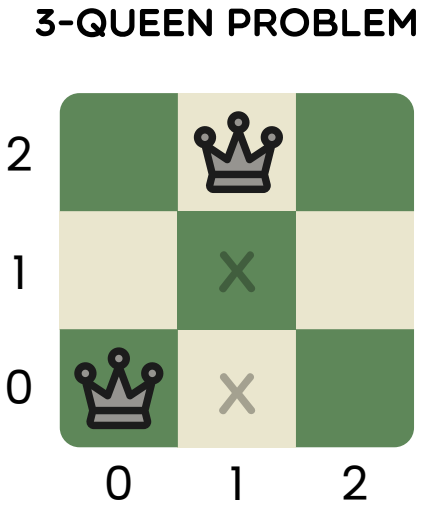


*Fig 1.2.52*

We now have a queen in column 0 and column 1, therefore Q3 must go in column 2. Can it go in square (2,0)? No Q1 is in that row. Can Q3 go in square (2,1)? No, it is threatened diagonally by Q2. Can Q3 go in square (2,2)? No that is the same row as Q2 and also threatened diagonally by Q1.

As our logic and Fig 1.2.53 shows there is no possible way to place a queen in the third column. Have we finished? Does this mean there are no solutions? Not yet.

We need to backtrack. This involves removing the previously placed queen, in this case Q2. We then look for other options for Q2 in column 1, all locations in column 1 have been tried, so we backtrack again. This time we remove Q1 and look for its next possible location.

We place Q1 in square (0,1) as shown in fig 1.2.54. Now we consider placing Q2 into column 1. Square (1,0) is diagonally threatened, (1,1) is horizontally threatened as it's in the same row, and square (1,2) is diagonally threatened by Q1.

As Fig 1.2.M shows, there are no locations that work in column 1 so we must backtrack again.

Q1 is removed from (0,1) and placed in square (0,2). We then consider column locations for Q2. Square (1,0) is a potential candidate so we place Q2 in that location and move on to locations for Q3 in column 2. As fig 1.2.55 shows No safe squares exist in column 2. Again we must backtrack.

We remove Q2 and try it one row up within column 1. We ask is square (1,1) safe? No, it is diagonally threatened by Q1 in (0,2). We backtrack, removing Q2 from (1,1). We ask, is square (1,2) safe? No, it is the same row as Q1. Therefore we backtrack again to remove Q2.

This takes us back considering a new location for Q1. There are no additional locations for Q1 as we have exhausted all possible locations in column 0.

Therefore, there are no solutions to the 3-Queen problem.

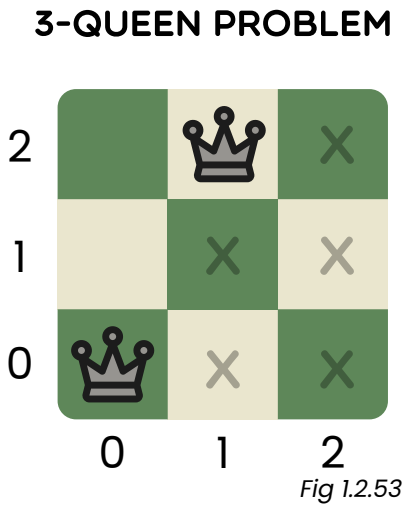


*Fig 1.2.53*

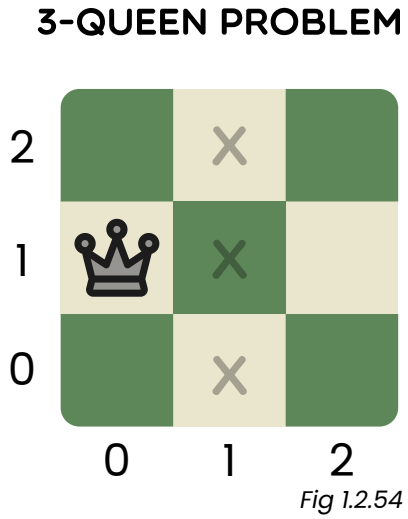


*Fig 1.2.54*

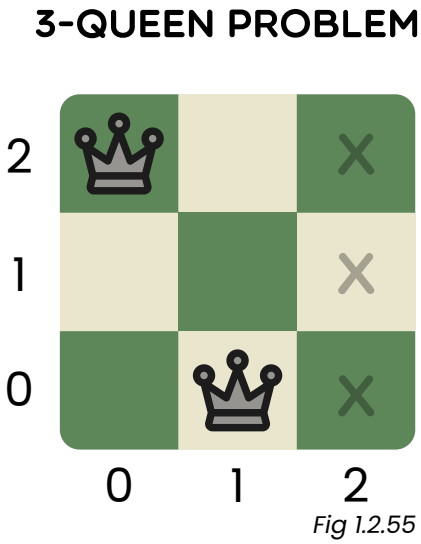


*Fig 1.2.55*

**N-Queens board configuration**

As N increases in size the problem space, or in probability theory, the sample space becomes factorially larger. As we are aware from our BigO notation work, factorial growth is significantly more rapid than exponential growth.

You will likely encounter the following combinations formula in your mathematics course.

$$^{n}C_{r} = \frac{n!}{(n-r)!r!}$$

For the N-Queen problem, in the formula we substitute $n=N^2$ and $r=N$. The following table, Fig 1.2.56 on page 128, shows the rapid increase in the number of unique board configurations as N increases compared to exponential growth $2^N$.

The number of possible unique board configurations for 3 queens on a 3 by 3 board is just 84, for 4 queens on a 4 by 4 board it is 1820 and on 5 queens on a 5 by 5 board the sample space increases to 53130. For an actual chess board measuring 8 by 8, there are over 4 billion possible configurations of 8 queens.

## CLASS ACTIVTY

**Activity: N-Queens Board Configurations**
Calculate on paper, without the aid of a calculator, the number of board configurations for the N=1, N=2, N=3 and N=4 cases to confirm the results shown in the table.

**Discuss: Issues with large N-Queens - code**
Assuming we are able to develop a correct coded solution for the general N-Queens problem. Discuss issues likely to occur should we attempt to use the code to solve the N-Queen problem for even moderately large values of N.

## INDIVIDUAL TASK

**Activity: 4-Queen problem**
Before reading on, work through the 4-Queen problem individually using the strategy outlined for the 3-Queen problem. Is there a solution?

## N-Queens Board Configurations

| N | Board Configurations | Exponential |
|---|---|---|
| 1 | 1 | 2 |
| 2 | 6 | 4 |
| 3 | 84 | 8 |
| 4 | 1820 | 16 |
| 5 | 53130 | 32 |
| 6 | 1947792 | 64 |
| 7 | 85900584 | 128 |
| 8 | 4426165368 | 256 |
| 9 | 260887834350 | 512 |
| 10 | 17310309456440 | 1024 |
| 11 | 1276749965026540 | 2048 |
| 12 | 103619293824707000 | 4096 |
| 13 | 9176358300744340000 | 8192 |
| 14 | 880530516383349000000 | 16384 |
| 15 | 91005567811177500000000 | 32768 |
| 16 | 10078751602022300000000000 | 65536 |
| 17 | 1190739044344490000000000000 | 131072 |
| 18 | 149482492334195000000000000000 | 262144 |
| 19 | 19870867053543800000000000000000 | 524288 |
| 20 | 2788360983670900000000000000000000 | 1048576 |
| 21 | 411887396336567000000000000000000000 | 2097152 |
| 22 | 63887407766986800000000000000000000000 | 4194304 |
| 23 | 10381758958529600000000000000000000000000 | 8388608 |
| 24 | 1763783520005150000000000000000000000000000 | 16777216 |
| 25 | 312690620414908000000000000000000000000000000 | 33554432 |
| 26 | 57746226578042000000000000000000000000000000000 | 67108864 |
| 27 | 11091107763254900000000000000000000000000000000000 | 134217728 |
| 28 | 2212202373592630000000000000000000000000000000000000 | 268435456 |
| 29 | 457589840052977000000000000000000000000000000000000000 | 536870912 |
| 30 | 98033481673745900000000000000000000000000000000000000000 | 1073741824 |

*Fig 1.2.56*

## 4-Queen problem

Consider N=4. This time we have sixteen squares to place 4 queens. It is now not immediately obvious whether a solution exists without a clear strategy, however if you completed the above activity you will know there is a solution, in fact there are two unique solutions.

**2 SOLUTIONS TO 4 QUEENS PROBLEM**

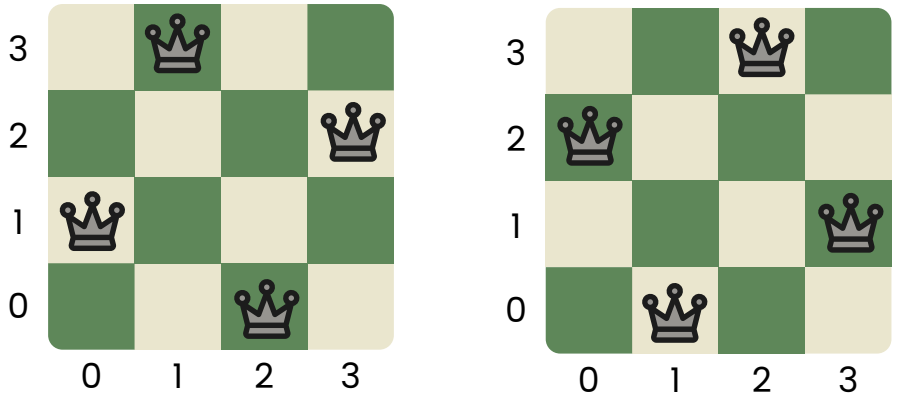



*Fig 1.2.57*

As we work through the 4-Queen version of the N-Queen problem we are going to focus on designing an algorithm based on the strategy we've used for the smaller versions of the problem.

First let us consider how we will represent the board. For this we need a data structure. A natural choice would be a 2-dimensional structure, a 2-dimensional array. We would then store a "Q" in the appropriate array element to indicate each location where a queen is currently placed.

If the array is called board, then to represent the first solution in Fig 1.2.57 above the following array elements would need to be set to "Q"; board(0,1), board(1,3), board(2,0) and board(3,2). All other array elements would be left empty.

### CLASS DISCUSSION

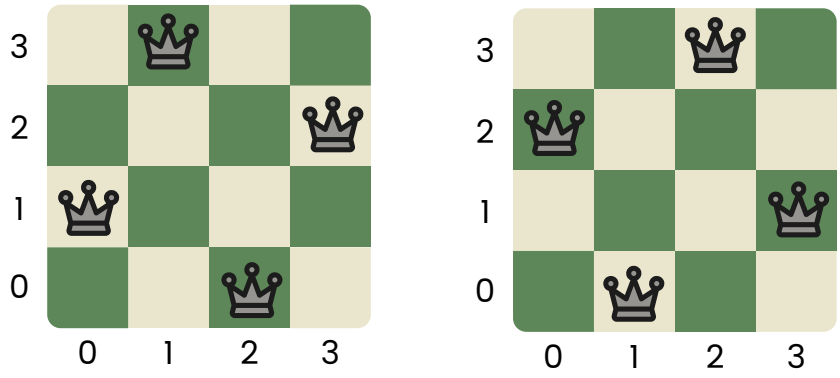
**Discuss: 2D array - 4-Queens**
Using the two-dimensional array above, which elements need to contain a Queen to represent the second solution?

Discuss issues that may be encountered writing an algorithm to solve the 4-Queens problem using this 2-dimensional array data structure.

Is there a better data structure?

Using the 2-dimensional array data structure, all elements apart from 4 are left empty. In addition you may have noticed that every arrangement of queens (not just for valid solutions) is of the form board(0,y0), board(1,y1), board(2,y2), board(3,y3). The real data is present in just 4 elements of the 2-dimensional array, so our array need only contain 4 elements.

A single 1-dimension array can do this. The index for the array indicates the column number from 0 to 3, and the data stored is the row number. Therefore, the two solutions would be represented as shown in Fig 1.2.58. In summary, the results 1302 and 2031 represent the two solutions to the 4-Queen problem.




**4-QUEENS SOLUTIONS USING A 1-DIMENSIONAL ARRAY**

| 1 | 3 | 0 | 2 |
|---|---|---|---|

| 2 | 0 | 3 | 1 |
|---|---|---|---|

*Fig 1.2.58*

Let's call the array board, and for the first solution is would be as follows.

board(0) = 1
board(1) = 3
board(2) = 0
board(3) = 2

This is great when we have a complete solution to represent, however during processing we may wish to indicate that no queen is located in a column. Let us use -1 to indicate a column is vacant. Therefore, a clear board would be represent as follows.

board(0) = -1
board(1) = -1
board(2) = -1
board(3) = -1

We will use this data structure to represent the current state of the board as we work on the design of our algorithm.

Let's review the logic of our solution strategy in general terms.

We begin by placing the first queen in the first square of the first column. We then check for safe locations for the second queen in the second column and each time we find a safe space we place the second queen and move on to looking for a place for the third queen in the third column. If we find a safe place we place the third queen and move onto looking for a safe place for the fourth queen. If successful placing all queens safely, then we have a solution. Each time we are unsuccessful in finding a safe space we backtrack, and consider the next possible square in our search.

There are four nested loops inherent in the above description, one iterates through each row in each column. And we only enter the next column's loop if we have found a safe location for a queen in the preceding loop.

Logically we remove the queen from a conflicting square and add it to the next square we wish to consider. As a result of our chosen data structure, moving a queen to a new row in a column simply requires changing the value in the appropriate array location, for example to move a queen from square (1,2) to square (1,3) we simply overwrite the current value of 2 held in board(1) with the value 3 – namely set board(1) = 3. If we wish to remove all queens from a column, such as occurs when we have exhausted the search of a column, we set board(col) = -1.

A possible algorithm could be.

**4- Queens algorithm: Pseudocode**

```
BEGIN fourQueens
        clearBoard
        FOR rowInCol0 = 0 TO 3
                board(0) = rowInCol0
                FOR rowInCol1 = 0 TO 3
                        board(1) = rowInCol1
                        IF isSafe(1) THEN
                                FOR rowInCol2 = 0 TO 3
                                        board(2) = rowInCol2
                                        IF isSafe(2) THEN
                                                FOR rowInCol3 = 0 TO 3
                                                        board(3) = rowInCol3
                                                        IF isSafe(3) THEN
                                                                outputBoard
                                                        ENDIF
                                                NEXT rowInCol3
                                                board(3) = -1
                                        ENDIF
                                NEXT rowInCol2
                                board(2) = -1
                        ENDIF
                NEXT rowInCol1
                board(1) = -1
        NEXT
END fourQueens
```

*Fig 1.2.59*

There are three subroutines called by the above algorithm, clearBoard, isSafe and outputBoard. The clearBoard subroutine sets all values in the board array to -1 as follows.

**Clear board: pseudocode**

```
BEGIN clearBoard
        FOR column = 0 TO 3
                Board(column) = -1
        NEXT column
END clearBoard
```

*Fig 1.2.60*

The isSafe routine is a function that takes as its input parameter the current column as input and it returns whether this queen is safe in relation to all queens in columns to the left. Note that there is no need to check all queens to the left are safe with regard to each other as this has already been assured.

We know that the current queen is safe in terms of columns as our algorithm is setup such that it is impossible to have more than one queen in a column. We need to check the row is unique and that there are no diagonal conflicts.

The row check is simple, if the row of any column to the left is equal to the row of the current queen then we have a conflict and isSafe should return false.

The diagonal check is a little more involved. Think of the grid as you would an x,y plot in maths, Consider the slope of the line drawn between the location of any two queens – the rise over the run, or the change in y over the change in x. If this slope is 1 (indicating 45 degrees) then the two queens are conflicted and isSafe should return false. Similarly, if the slope is -1 (indicating -45 degrees) they are also conflicted.

We can use the absolute value of the rise and run to combine both diagonal checks into one.

A simpler way of asking does rise over run equal one, is to ask does rise equal run? In our case does the absolute value of the difference in rows equal the absolute value of the difference in columns? If the answer is yes, then we have a conflict and the queen is not safe.

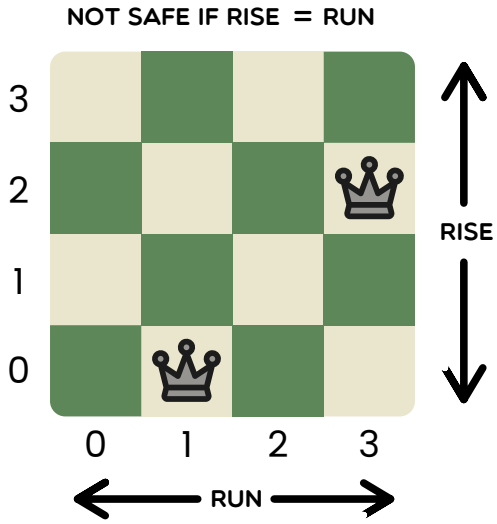


*Fig 1.2.61*

**Is Safe: pseudocode**

```
BEGIN isSafe(c)
        safeSpace = True
        FOR i = 0 TO c-1
                IF board(i) = board(c) THEN
                        safeSpace = False
                ENDIF
                IF Abs(board(c) - board(i)) = Abs(c - i) THEN
                        safeSpace = False
                ENDIF
        NEXT i
        RETURN safeSpace
END isSafe
```

*Fig 1.2.62*

We could output the board in a range of different ways. For now, let's just simply output the elements of the board array on a single line.

**Output Board: pseudocode**

```
BEGIN outputBoard
        FOR I = 0 TO 3
                Print board(i) " "
        NEXT i
        Print to new line
END outputBoard
```

*Fig 1.2.63*

Following is a desk check of the above algorithm.

| rowInCol0 | rowInCol1 | rowInCol2 | rowInCol3 | board(0) | board(1) | board(2) | board(3) | isSafe | output |
|---|---|---|---|---|---|---|---|---|---|
| | | | | -1 | -1 | -1 | -1 | | |
| 0 | | | | 0 | | | | | |
| | 0 | | | | 0 | | | FALSE | |
| | 1 | | | | 1 | | | FALSE | |
| | 2 | | | | 2 | | | TRUE | |
| | | 0 | | | | 0 | | FALSE | |
| | | 1 | | | | 1 | | FALSE | |
| | | 2 | | | | 2 | | FALSE | |
| | | 3 | | | | 3 | | FALSE | |
| | | | | | | -1 | | | |
| | 3 | | | | 3 | | | TRUE | |
| | | 0 | | | | 0 | | FALSE | |
| | | 1 | | | | 1 | | TRUE | |
| | | | 0 | | | | 0 | FALSE | |
| | | | 1 | | | | 1 | FALSE | |
| | | | 2 | | | | 2 | FALSE | |
| | | | 3 | | | | 3 | FALSE | |
| | | | | | | | -1 | | |
| | | 2 | | | | 2 | | FALSE | |
| | | 3 | | | | 3 | | FALSE | |
| | | | | | | -1 | | | |
| | | -1 | | | | | | | |
| | -1 | | | | | | | | |
| 1 | | | | 1 | | | | | |
| | 0 | | | | 0 | | | FALSE | |
| | 1 | | | | 1 | | | FALSE | |
| | 2 | | | | 2 | | | FALSE | |
| | 3 | | | | 3 | | | TRUE | |
| | | 0 | | | | 0 | | TRUE | |
| | | | 0 | | | | 0 | FALSE | |
| | | | 1 | | | | 1 | FALSE | |
| | | | 2 | | | | 2 | TRUE | 1302 |
| | | | 3 | | | | 3 | FALSE | |
| | | | | | | | -1 | | |
| | | 1 | | | | 1 | | FALSE | |
| | | 2 | | | | 2 | | FALSE | |
| | | 3 | | | | 3 | | FALSE | |
| | | | | | | -1 | | | |
| | | | | | -1 | | | | |

| rowInCol0 | rowInCol1 | rowInCol2 | rowInCol3 | board(0) | board(1) | board(2) | board(3) | isSafe | output |
|---|---|---|---|---|---|---|---|---|---|
| 2 | | | | 2 | | | | | |
| | 0 | | | | 0 | | | TRUE | |
| | | 0 | | | | 0 | | FALSE | |
| | | 1 | | | | 1 | | FALSE | |
| | | 2 | | | | 2 | | FALSE | |
| | | 3 | | | | 3 | | TRUE | |
| | | | 0 | | | | 0 | FALSE | |
| | | | 1 | | | | 1 | TRUE | 2031 |
| | | | 2 | | | | 2 | FALSE | |
| | | | 3 | | | | 3 | FALSE | |
| | | | | | | | -1 | | |
| | | | | | | -1 | | | |
| | 1 | | | | 1 | | | FALSE | |
| | 2 | | | | 2 | | | FALSE | |
| | 3 | | | | 3 | | | FALSE | |
| | | | | | -1 | | | | |
| 3 | | | | 3 | | | | | |
| | 0 | | | | 0 | | | TRUE | |
| | | 0 | | | | 0 | | FALSE | |
| | | 1 | | | | 1 | | FALSE | |
| | | 2 | | | | 2 | | TRUE | |
| | | | 0 | | | | 0 | FALSE | |
| | | | 1 | | | | 1 | FALSE | |
| | | | 2 | | | | 2 | FALSE | |
| | | | 3 | | | | 3 | FALSE | |
| | | | | | | | -1 | | |
| | | 3 | | | | 3 | | FALSE | |
| | | | | | | -1 | | | |
| | 1 | | | | 1 | | | TRUE | |
| | | 0 | | | | 0 | | FALSE | |
| | | 1 | | | | 1 | | FALSE | |
| | | 2 | | | | 2 | | FALSE | |
| | | 3 | | | | 3 | | FALSE | |
| | | | | | | -1 | | | |
| | 2 | | | | 2 | | | FALSE | |
| | 3 | | | | 3 | | | FALSE | |
| | | | | | -1 | | | | |

*Fig 1.2.64*

CLASS DISCUSSION

**Activity: 4-Queens desk check**
Work through the algorithm and desk check alongside a 4 by 4 grid on a whiteboard. This enables you to easily wipe off and add queens as you work through the algorithm. Take particular notice when you are backtracking.

**Discuss: Desk check observations**
Some observations to discuss based on the desk check…

1. There are just 56 True/False entries in the isSafe column. Does this mean our algorithm only needed to test 56 of the 1820 possible ways to place 4 queens into a 4 by 4 grid? Discuss.

2. Were there any checks done that could have been avoided? Suggest enhancements to the strategy we have used to further improve the efficiency of the algorithm.

3. What is the time complexity of the above algorithm? Discuss.

## 8-Queens and N-Queens problem

Now let us consider the original 8-Queens and N-Queens problems.

Could we extend the above algorithm for the 4-Queens problem to 8-Queens? Yes, it would involve adding a further 4 loops to cater for columns numbered 4 through to 7. A rather messy solution, but it would work.

What about a general N-Queens solution? A solution where N is an input parameter? In that case we don't know in advance how many loops we need to code. Is there a better way?

The following is a recursive solution to the N-Queens problem.

```
BEGIN nQueens(n)
     clearBoard
     rQueens(0)
END nQueens

BEGIN rQueens(col)
     IF col = n THEN
          outputBoard
     ELSE
          FOR row = 0 TO n - 1
               Board(col) = row
               IF isSafe(col) THEN
                    rQueens(col + 1)
               ENDIF
          NEXT row
     ENDIF
END rQueens
```

*Fig 1.2.65*

In the above algorithm, the clearBoard, outputBoard and isSafe subroutines will be essentially the same as those used as part of the previous iterative fourQueens solution.

Fig 1.2.66 below shows the call tree for the n=4 case, that is nQueens(4). The red ordered pairs are included to indicate each time a queen was added to the board. Working back up the branches of the tree, the red ordered pairs identify the location of every queen currently on the board that are considered during calls to isSafe.

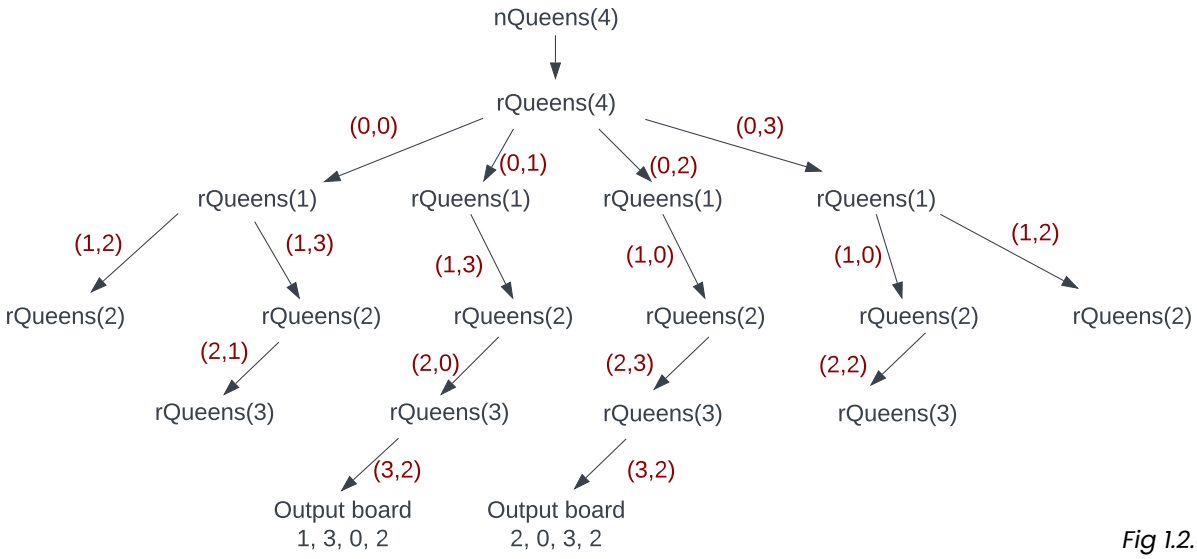



*Fig 1.2.66*

## CLASS DISCUSSION

**Activity: 4-Queens & 5-Queens**
Work through the call nQueens(4) with a 4 by 4 grid on a whiteboard so you can easily move queens as needed. Do your results agree with Fig 2.1.66?

There are 10 solutions for the 5-Queens problem. Use the recursive algorithm to determine the solutions. You might like to assign different class members different starting positions for the column 0 queen to reduce the effort.

**Discuss: nQueens**
Why is there an nQueens routine that calls the rQueens routine? Discuss.

Each red ordered pair in Fig 2.1.66 indicates a queen being placed on the board in a safe location relative to queens in columns to its left. Confirm this is the case.

Do you think the number of queen placements is an indication of the efficiency of the algorithm?

What is the time complexity of the recursive N-Queens algorithm above? Discuss.

## EXERCISE 1.2.4 - MULTIPLE CHOICE

1. Which of the following is the most efficient sort?

   A. Permutation sort
   B. Selection sort
   C. Binary sort
   D. Merge sort

2. To sort 10 items using the permutation sort described in this textbook is likely to take approximately how many iterations?

   A. 100
   B. 1,000
   C. 100,000
   D. > 1,000,000

3. If there are 1000 items in an array, what is the approximate average number of comparisons required to find the largest item at the start of a selection sort?

   A. 10
   B. 100
   C. 500
   D. 1000

4. If an array if n items is sorted, what is the time complexity to find the largest item?

   A. O(1)
   B. O(log n)
   C. O(n)
   D. O(n log n)

5. The algorithm design strategy known as divide and conquer is used by which of the following algorithms?

   A. Linear Search and Selection Sort
   B. Binary Search and Selection Sort
   C. Linear Search and Merge Sort
   D. Binary Search and Merge Sort

6. An ascending selection sort could be performed using which strategy during each pass through the array?

   A. Find the largest unsorted item and swap with the last unsorted item.
   B. Find the smallest unsorted item and swap with the first unsorted item.
   C. Find the smallest unsorted item and swap with the last unsorted item.
   D. Either A or B.

7. What is the result of the two assignment statements X=Y followed by Y=X?

   A. The values in X and Y are swapped.
   B. Both X and Y are set to the initial value of X.
   C. Both X and Y are set to the initial value of Y.
   D. There is no change in either X or Y values.

8. Which of the following best describes the merge process within a Merge Sort?

   A. Each of two smaller arrays are sorted into order.
   B. A larger unsorted array is split into two smaller unsorted arrays.
   C. Two smaller sorted arrays are combined to form a larger sorted array.
   D. Arrays with a single element are combined to form a series of sorted arrays that each contain 2 elements.

9. Which of the following best describes the divide process within a Merge Sort?

   A. Each of two smaller arrays are sorted into order.
   B. A larger unsorted array is split into two smaller unsorted arrays.
   C. Two smaller sorted arrays are combined to form a larger sorted array.
   D. Arrays with a single element are combined to form a series of sorted arrays that each contain 2 elements.

10. For a 5-Queen problem, using the data structure described in the text, how would the following solution be represented as an array?

| | | | | |
|---|---|---|---|---|
| Q | | | | |
| | | | Q | |
| | Q | | | |
| | | | | Q |
| | | Q | | |

   A. 4, 2, 0, 3, 1
   B. 0, 3, 1, 4, 2
   C. 5, 3, 1, 4, 2
   D. 1, 4, 2, 5, 3

## EXERCISE 1.2.4 - SHORT ANSWER

11. Write down 10 names of 10 family members or friends on slips of paper. Shuffle and sort them using a selection sort, then do the same using a merge sort. Now explain how a permutation sort works to one of your family and friends and ask them to sort the slips using a permutation sort strategy.

12. What changes need to be made to the selection sort algorithms in the text to sort the items into descending rather than ascending order?

13. In your own words, describe the general divide and conquer strategy using the merge sort as an example.

14. In your own words, describe the general back tracking strategy using the N-Queens problem as an example.

15. An existing array contains a sorted list of all of ID numbers. Every day or so, a new ID needs to be added to this array in the correct position. Design an algorithm that will accomplish this task.

## Programming Paradigms

Different programming paradigms have emerged to address different programming needs and to provide alternative approaches to software development. They have emerged due to a variety of factors, including advancements in technology, changes in programming languages, evolving software requirements, and the desire for more efficient and expressive ways of solving problems.

### Object-oriented programming

Object-Oriented Programming (widely known as OOP) is a programming paradigm that organises code into objects, which are instances of classes. It emphasises the concepts of encapsulation, inheritance, and polymorphism. OOP focuses on modelling real-world entities as objects with properties (attributes) and behaviours (methods). It enables modular and reusable code using classes, objects, and their interactions.

OOP is used in the development of complex software systems that require modular design, code reuse, and the option for future extension or "extensibility". OOP is well-suited for modelling real-world entities and their interactions. Examples include creating GUI applications with graphical components, designing video games with various game objects and behaviours, or building simulations that mimic real-world scenarios.

Python, along with other popular Object-Oriented Programming (OOP) languages such as Java, C++, C#, Ruby, and JavaScript, dominates the programming landscape. Among these languages, Python stands out for its simplicity, and readability. It excels in various domains, including web development, scientific computing, and data analysis. Python's clean syntax and numerous libraries make it an ideal language for both beginners and experienced developers, providing many opportunities to apply OOP principles and build modular and maintainable code. Furthermore, Python benefits from a large and active community, ensuring comprehensive learning resources and strong community support.

While each language has its strengths, Java is renowned for its platform independence and robustness, C++ for its high performance and low-level control, C# for Windows application development, Ruby for its elegant syntax and web development with Ruby on Rails, and JavaScript for its dominance in web development.

You will engage with Python's OOP capabilities throughout the Software Engineering course, in particular in Chapter 2.1 and 2.2 so we will not delve further into OOP at this stage. You are encouraged to experiment with a wide range of OOP languages to suit different development needs.

#### CLASS RESEARCH

**Research: Object Oriented Programming Features**
Each student is to research examples of OOP code in a particular language. Ensure a range of languages are examined. Obtain simple examples of code for defining a class, and instantiating an object. Share your findings with the class.

**Discussion: Benefits of Objects**
The combining of data and methods into objects is central to OOP. How does this assist modular design, code reuse and extensibility?

## Imperative

Imperative programming is a form of procedural programming paradigm where the focus is on explicitly specifying the steps required to perform a task. The algorithms we have considered so far, follow this premise. It emphasises changing the program's state through a sequence of instructions and control structures like loops and conditionals. Imperative programming languages provide explicit commands for manipulating variables and data, allowing programmers to specify how the program should execute.

Imperative programming is used when writing programs that follow a step-by-step approach and explicitly define the program's flow. Imperative programming is commonly used for system-level programming, low-level hardware interactions, and algorithm implementations. Most beginner programmers will start their learning journey working in this paradigm. Examples include writing procedural code to manage system resources, implementing algorithms such as sorting or searching, or developing programs that involve sequential instructions and control structures.

**Object Oriented Paradigm**

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def calculate_area(self):
        return 3.14 * self.radius ** 2

    def calculate_circumference(self):
        return 2 * 3.14 * self.radius

# Create an instance of the Circle class
my_circle = Circle(5)

# Calculate and print the area and cir-
cumference of the circle
print("Area:", my_circle.calculate_
area())
print("Circumference:", my_circle.calcu-
late_circumference())
```

**Imperative**

```
def calculate_area(radius):
    return 3.14 * radius ** 2

def calculate_circumference(radius):
    return 2 * 3.14 * radius

# Define the radius of the circle
radius = 5

# Calculate and print the area and cir-
cumference of the circle using functions
area = calculate_area(radius)
circumference = calculate_circumfer-
ence(radius)
print("Area:", area)
print("Circumference:", circumference)
```

Fig 1.2.67 - Drawing a circle in Python

In the above examples, the first code snippet demonstrates an OOP style by defining a Circle class with attributes (radius) and methods (calculate_area and calculate_circumference). An instance of the class is created, and the methods are called on that instance to calculate and print the area and circumference of the circle.

The second code snippet follows an Imperative style by using functions to calculate the area and circumference of a circle. The radius is defined as a variable, and the functions are called with the radius value to compute and print the area and circumference.

CLASS DISCUSSION

**Compare: OOP vs Imperative**
Compare and contrast the OOP and Imperative Python code snippets in Fig 2.1.x. Identify aspects of each snippet that indicate OOP and aspects that indicate an imperative paradigm.

## Logic

Logic programming is a declarative programming paradigm where the emphasis is on describing the problem's logic rather than the control flow. It is based on formal logic and involves defining a set of logical rules and facts. Programs written in logic programming languages specify what needs to be achieved, and the execution engine resolves the logic and generates the desired results.

It is suited to the development of programs that rely on logical rules and facts to solve problems. Logic programming is often used for rule-based systems, expert systems, and applications that involve reasoning and inference. The logic paradigm is used for building rule-based systems for decision support, creating expert systems that provide specialised knowledge and recommendations, or developing AI applications that require logical reasoning and pattern matching.

There is significant year 12 content looking at machine learning and artificial intelligence which is heavily influences by the logic programming paradigm.

CLASS ACTIVITY

**Investigate: Expert systems**
Expert systems aim to reproduce the expertise of a human expert. Find examples of expert systems that are used in medicine, finance, and engineering fields.

- Identify the purpose of each system.
- Outline any limitations on the advice or output each system is able to provide.

**Activity: Online expert system creation**
Use an online expert system tool to build a simple expert system. Some ideas include:

- diagnosing a common cold vs the flu vs seasonal allergies
- identify a type of rock, or bird or animal
- determining the make, model and variant of motor vehicle, agricultural implement, ship or aircraft.

## Introduction to Prolog by example

Let us assume we have a simple food chain where lions eat dogs, dogs eat cats, cats eat birds, birds eat spiders and spiders eat flies. In Prolog we write the following facts:

```
eat(lion,dog).
eat(dog,cat).
eat(cat,bird).
eat(bird,spider).
eat(spider,fly).
```

In Prolog eat is called a predicate; to be more precise it is a user defined predicate. We just made up the name eat; we could have used devour or almost any other name. The animal's names are objects. These are simple objects that are known in Prolog as atoms. Atoms must commence with a lower case character. A fact is a definite known item e.g. cats eat birds or in Prolog eat(cat, bird)

### CLASS ACTIVITY

**Activity: Facts in prolog**
Design a set of facts, using Prolog syntax, describing the components of a computer system as either input, processing or output components. Remember that some components are both input and output devices.

Querying these facts is simple. Say we wish to determine if a spider can eat a dog. We enter the line ?-eat(spider,dog). When we run our program (or query) the Prolog compiler responds with No where as if we enter the query ?eat(dog,cat)the output is Yes. In Prolog a query is known as a goal, i.e. our goal was to find out if a dog could eat a cat. When we just have facts in our database and no rules then Prolog merely searches through the facts for a match. If a match occurs then our goal has been fulfilled and "Yes" is output. If no match is found then our goal fails and "No" is output.

**Key Term: Goal**
A query that can result in either being fulfilled, in which case the result is Yes, or not being fulfilled, in which case the result is No.

### CLASS DISCUSSION

**Discuss: Goals**
Write down two goals that will be fulfilled and two goals that will not. What do you think would be the result of the following goals?

? eat(pig,table)

? eat(dog,bird)

Let us introduce a general fact into our program to say that everyone likes eating flies. Rather than having to write a long list of new facts we can use a variable, say X to represent any value. Our new fact would be eat(X,fly). Variables must have an upper case first letter. Now if our goal is ?eat(dog, fly) or ?eat(lion,fly) or ?eat(egg, fly) the result will always be true.

Let us further generalise by introducing a rule into our system. Say that if a lion can eat a dog and a dog can eat a cat then a lion must be able to eat a cat. By continually applying this rule we see that all animals will be able to eat flies so we can remove eat(X,fly) from our facts. Generalising this statement we get something like this: If A can eat B and B can eat C then A can eat C, in Prolog we write this as: eat(A,C):-eat(A,B),eat(B,C). Notice that A, B and C are in upper case so they are variables. Facts and rules combine to form a knowledge base. During execution of a goal Prolog interrogates the knowledge base in an attempt to fulfil the goal.

As an example we wish to know if dogs eat spiders. Executing the goal ?eat(dog,spider) yields the result Yes as expected. How is Prolog executing this goal? Firstly the list of facts is examined in search of a direct match; none is found. Next the rules are examined. In our example only one rule exists. This rule allows us to reduce our goal into two sub-goals, eat(dog,X) and eat(X,spider). We then examine eat(dog,X) and compare it to our facts. The first, and in this case the only match is eat(dog,cat), the sub-goal eat(dog,X) is fulfilled. For our original goal to be fulfilled we require the sub-goal eat(cat,spider) to be fulfilled. Breaking this goal down using our rule we get eat(cat,Y) and eat(Y,spider). Examining our facts we find that eat(cat,bird) fulfils the first of these goals and eat(bird,spider) fulfils the second. As a result of these inferences the original goal is fulfilled and Yes is output.

**Key Term: Knowledge Base**
In terms of logical programming, a knowledge base is a database containing all the facts and rules.

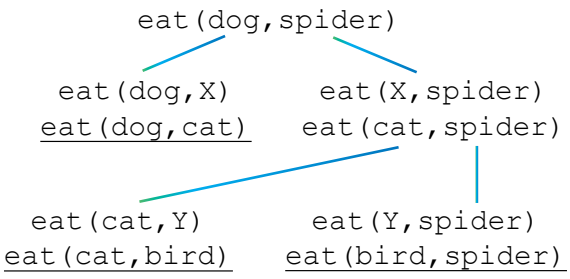



*Fig 1.2.68*
*Tree showing a trace of the fulfilment of a goal.*

**Key Term: Inference engine**
The control mechanism that applies knowledge, contained in a knowledge base, to resolve goals. The result being fulfilment or failure of the goal.

The inference engine carries out this processing. The inference engine is the processing unit of logical programming languages. Inference engines apply the knowledge contained within the knowledge base to reach conclusions about goals. They do this in an organised and systematic manner.

## CLASS DISCUSSION

**Activity: Draw trace tree**
Draw a trace tree, like the one in Fig 1.2.68, for each of the following goals.

? eat(lion,bird)

? eat(cat,fly)

The goal ?eat(bird,dog) should result in an output of No. Actually, Prolog reports a stack overflow error. Let us examine the way Prolog's inference engine approaches the resolution of this goal. As eat(bird,dog) is not a fact within the knowledge base we split into sub-goals using our rule. As a consequence we have eat(bird,A) and eat(A,dog). We need to fulfil eat(bird,A). There is a fact eat(bird,spider) that fulfils this goal. As A could be spider we now try to resolve eat(spider,dog).

```
eat(lion,dog).
eat(dog,cat).
eat(cat,bird).
eat(bird,spider).
eat(spider,fly).
eat(A,C):-eat(A,B),eat(B,C).
```

*Fig 1.2.69*
*Prolog code showing our knowledge base. Note that each fact or rule ends with a full stop.*

This is not a fact so again we use our rule. We need to resolve eat(spider,B) and eat(B,dog). We find a fact, eat(spider,fly) that fulfils the first of these sub-goals, so B could be fly. We need to resolve eat(fly,dog) which is not a fact so we once again use our rule resulting in eat(fly,C) and eat(C,dog). No fact exists for eat(fly,C) so again we use our rule which gives us the sub-goals eat(fly,D) and eat(D,C). Again no facts for eat(fly,D) so we apply our rule giving us eat(fly,E) and eat(E,D). This process continues infinitely and eventually causes a stack overflow error.

This is a common problem encountered in Prolog when rules contain themselves – it is another example of recursion. Prolog compilers and interpreters have mechanisms to detect these infinite inferences occurring, remember however, that logically they should occur. We need a base case that can be solved without recursion.

A work around is to create a new rule based on the old rule. In our example code we'll call this new rule caneat where caneat(dog,spider) means a dog can eat a spider. Following is our corrected rule:

```
caneat(A,C):-eat(A,C).
caneat(A,C):- eat(A,B),caneat(B,C).
```

Let us examine our new goal question, ?caneat(bird,dog). The goal does not match any facts so we examine our two rules. Using the first rule we get eat(bird,dog). This is not a fact and cannot be broken down. Examining our second rule gives us eat(bird,X) and caneat(X,dog). The facts indicate that X could be spider, as eat(bird,spider) is a fact, in which case we need to resolve caneat(spider,dog). Again this is not a fact and using our first rule neither is eat(spider,dog). Our second rule results in eat(spider,Y) and caneat(Y,dog). Y could be fly as spiders eat flies, so we try to resolve caneat(fly,dog). This goal expands to eat(fly,Z) and caneat(Z,dog). So far the operations of the inference engine have been similar to our original example above. This is the point at which the addition of our new rule affects the operation of the inference engine. No fact or rule exists in our knowledge base to resolve eat(fly,Z) any more. As a consequence the output is the coveted result No. Which means birds cannot eat dogs. Phew! All that work for such an obvious result.

## CLASS DISCUSSION

**Activity: Prolog goal**
Work through the goal ? eat(lion,cat)to ensure our modified Prolog code still gives the correct output of Yes.
**Discuss: Prolog code logic**
How has our modified code solved the problem? Surely our code is still logically the same as the original? Discuss.

There are two common strategies used to resolve goals; backward chaining and forward chaining. So far our examples have used a backward chaining strategy. We started with a goal and sequentially searched through our knowledge base in search of facts and then rules that could be used to prove our goal. If we found a fact then the goal was proved. If we found a rule then we commenced our search once again at the start of the knowledge base using the goals within the rule. This process continued until we either proved our goal or reached a dead end.

**Key Term: Backward Chaining**
Assume the theory is true and then ask questions to systematically verify the necessary rules are present.

Forward chaining uses almost the reverse strategy to backward chaining. When using a forward chaining strategy we need to know some initial facts are true. We use these facts to reach a conclusion. It is possible for forward chaining to result in more than one conclusion. Prolog is able to use a combination of backwards and forward chaining; it depends on the question and the facts and rules held in the knowledge base. Let us leave Prolog now and consider a conceptual example to illustrate both backwards and forwards chaining.

**Key Term: Forward Chaining**
Start from the beginning of the facts and rules and ask questions to determine which path to follow next to arrive at a conclusion.

## CLASS DISCUSSION

**Aircraft Rules**

We have the following set of rules in regard to aircraft that we know to be true:

- Rule 1. If a plane has a jet engine and a single seat then it is a jet fighter
- Rule 2. If a plane has a jet engine and a pressurised cabin then it can fly above 15000 feet
- Rule 3. If a plane has fixed windows then it has a pressurised cabin
- Rule 4. If a plane can fly above 15000 feet then it must be air-conditioned

Suppose we wish to prove using these rules, that a jet with fixed windows must be air-conditioned. Let us examine this problem using firstly a backward chaining strategy and then using a forward chaining strategy:

**Backward chaining**

Using this method we commence with our desired goal, namely to prove that a jet with fixed windows must be air-conditioned. We scan down our rules until we find one that results in an air-conditioned plane. Rule 4 meets our needs. This rule provides us with a sub-goal to prove, namely that our plane can fly above 15000 feet. In other words if we can prove our plane can fly above 15000 feet our supposition must be true. We now scan our rules looking for one that results in a plane that flies above 15000 feet. Rule 2 meets this requirement. Two new sub-goals result; the plane must have a jet engine and it must have a pressurised cabin. We know the plane has a jet engine, this was part of our question but does it have a pressurised cabin? Rule 3 proves it does; any plane with fixed windows has a pressurised cabin, hence our plane must have a pressurised cabin because we know it has fixed windows. Our theory has been proved: jets with fixed windows must be air-conditioned.

**Forward chaining**

Using this approach we use the knowledge that our jet has fixed widows and attempt to reach a conclusion. Hopefully our conclusion will be that the plane must be air-conditioned. Scanning our list we find that Rule 3 means our plane must also be pressurized because it has fixed windows. We now know our plane is a jet with fixed windows that is pressurised. Using this information we scan our rules again. We find that Rule 2 is resolved correctly, so our plane can fly above 15000 feet. As a consequence Rule 4 becomes relevant and our plane must be air-conditioned. Our theory has once again been proved.

**Discussion: Forward Chaining**
Forward chaining can lead to more than one conclusion. Why is this the case? Discuss. Use our aircraft rules to point out an example where this could occur.

## Functional

Functional Programming is a paradigm that approaches computation as the evaluation of mathematical functions. In functional programming, programs are constructed by defining and composing functions rather than changing state or executing imperative commands. It focuses on immutability, avoiding side effects, and leveraging higher-order functions to manipulate data.

- **Immutability**:
  Immutability refers to the concept of making data objects unchangeable once they are created. In functional programming, data is treated as immutable, meaning it cannot be modified after it is created. Instead, functions operate on data to produce new values without modifying the original data. This promotes code clarity, predictability, and facilitates reasoning about program behaviour.

- **Avoiding Side Effects**:
  Side effects are modifications to the state of a system outside the function's scope. Side effects include changing variables, performing input/output operations, or modifying shared data. By minimizing or eliminating side effects, functional programming makes programs more reliable, testable, and easier to understand and reason about.

- **Higher-Order Functions**:
  Higher-order functions are functions that can take other functions as arguments or return functions as results. They allow for more expressive and flexible code by enabling operations on functions themselves. Higher-order functions can be used to isolate common patterns, create reusable code, and implement powerful functional programming techniques such as function composition and currying.

- **First-Class Functions**:
  Functions are, naturally, the key to functional programming in that they can be assigned to variables, passed as arguments to other functions, and returned as values. First-class functions provide a higher level of abstraction, enabling functional programming techniques like function composition and the creation of more modular and reusable code.

- **Lambda Expressions**:
  Lambda expressions, also known as anonymous functions, are concise and inline function definitions that do not require a separate named function. That is, they are defined and utilised directly within a line of code as needed, often as used small, ad-hoc functions in combination with higher-order functions. Lambda expressions provide a convenient way to express simple computations without the need for explicit function declarations.

- **Recursion**:
  In functional programming, recursion is often preferred over iterative loops as the primary control flow mechanism. It allows for more concise code by breaking down complex problems into simpler subproblems that leverage the concept of self-reference. Recursive functions can solve problems by breaking them down into smaller instances until a base case is reached.

Functional programming languages, such as Haskell, Lisp, APL, Scala, and Clojure, provide built-in features and syntax to support these functional programming principles and techniques. By embracing immutability, avoiding side effects, and leveraging higher-order functions, functional programming encourages code that is more modular, reusable, and easier to reason about. It provides a different approach to software engineering, promoting clean and concise code that emphasizes the evaluation of mathematical functions and the transformation of data.

## CLASS DISCUSSION

**Discuss: Functional programming**
All programming languages include functions, so what is so special about purely functional programming languages? Discuss with respect to immutability, side effects, higher order functions and first class functions.

Let us now consider Haskell and Lisp with brief examples.

### Haskell

Haskell is a pure functional language. Programs in Haskell are actually large functions. Haskell is named after Haskell Brooks Curry whose work with mathematical logic provided a solid basis for the creation of functional languages. Haskell is viewed as a specialised language relevant only to artificial intelligence applications. Actually Haskell is able to complete most programming tasks very well. The source code for a typical Haskell program is generally three to ten times shorter than its imperative source code equivalent. The syntax of Haskell looks very much like mathematical function notation.

Earlier we examined three sorts, permutation, selection and merge sort. There are of course many other ways of sorting. One common method is the quicksort.

Following is a Haskell function that performs a quicksort:

```
qsort []     = []
qsort (x:xs)        = qsort leftElements ++ [x] ++ rightElements
        where
                leftElements        = [y | y <- xs, y < x]
                rightElements       = [y | y <- xs, y >= x]
```

*Fig 1.2.70*

The first line of the above code means, "the result of sorting an empty list is an empty list". This seems a bit trivial, however this line is vital to the solution. The left hand side of the second line means; "to sort a list whose first element is x, and whose remaining elements are xs, do the following". The right hand side of the statement means "sort all the leftElements, sort all the rightElements and then concatenate (++) the results with the first element x in the middle". The remaining lines describe what the left and right elements are. The fourth line means "leftElements is a list of all y's such that y is a member of xs and y is less than the first element x". The definition of rightElements is similar however the values must be greater than or equal to x.

Our quicksort function uses recursion; the qsort function includes a call to the qsort function. When using functional languages this is the main method of achieving repetition. Recursion is a difficult process to deskcheck as many instances of the same identifiers are in existence at the same time. Nevertheless without recursion truly functional languages would not be able to perform repetition. The use of recursion is one of the major reasons why functional code is significantly shorter compared to the equivalent imperative code. To implement a quicksort in an imperative language would take up to ten times the number of lines than the functional equivalent.

Let's say we call the Haskell qsort function with the list 5, 3, 6, 4, 5, 3, 1, 8, 7, 2.

The first call splits the list into three pieces, those elements less than 17, 17 and those elements greater than or equal to 17. In Fig 1.2.71 this initial call is represented by the largest rectangle. Notice that 17 is in its correct position relative to all the other list elements. Within this first call are two recursive calls to qsort, one on the leftElements and another on the rightElements.

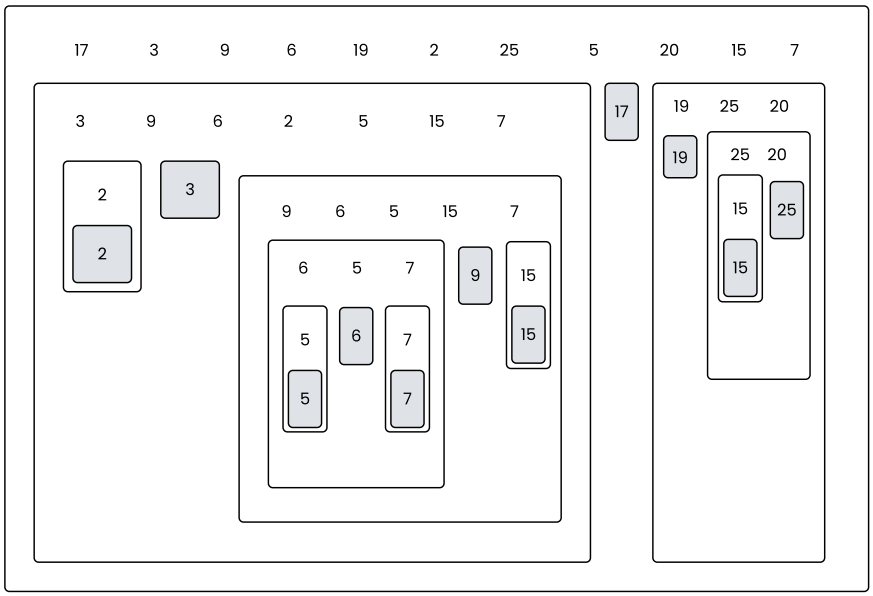



Fig 2.1.71
The Haskell implementation of a quicksort. Each recursive call is shown in a rectangle. Shaded items are in their final positions.

If we examine the leftElements call we find it creates a new set of leftElements and rightElements with the list value 3 in the middle. A recursive call then commences with these new versions of leftElements and rightElements. Now we consider the call to qsort using this new leftElements list. This time there is only one value in the list, namely 2. So our resulting three pieces are composed of an empty list, 2 and another empty list. The next calls to qsort, (with empty lists), are executed by the first line of our qsort function. This line does not contain a recursive call, rather it does nothing but return an empty list []. This is our base case and without it the process of recursion would continue and eventually cause a

stack overflow error. As a result of the base case, qsort is at last, able to return something. As both calls return empty lists the next higher-level qsort call is also able to end. On our diagram, see Fig 1.2.71, the conclusion of a call is indicated by the closing of the bottom of a rectangle. The conclusion of a function call to qsort means the result can be returned to the calling function. Once the result of both calls are received then that call can also close and send its result back to its calling function. This process continues until all function calls have been completed. Finally, the initial qsort call will be concluded and the sorted list will be returned to the source of the original call.

Our Haskell quicksort above will work just as well with lists of characters, strings, or real numbers. In fact it will work with lists of anything so long as you are able to determine if one element of the list is greater than another.

### Lisp

The basics of Lisp where developed by John McCarthy at Stanford University in 1958. In 1995, ANSI common Lisp was released. Common Lisp combines the features of an object oriented language with those of a functional language. Traditionally Lisp was an interpreted language, this is no longer the case. Most versions of Lisp come with both interpreters and compilers. Lisp stands for LISt Processing as Lisp is almost entirely about creating and manipulating lists of data items. The source code of a Lisp program is even a list! So what is a list? A list is a sequence of items, just like a shopping list.

Let us examine a simple Lisp list (+ 2 3). As you've probably surmised, when this list is evaluated the result is 5. What makes this a list are the brackets. In Lisp anything surrounded by brackets is a list. In this list, (+ 2 3), we have a function together with arguments. + is the function and 2 3 are the arguments. Functions always come first, as lists are evaluated from left to right. This is one feature of Lisp that makes it easy to learn; no matter what the function is, it will always come before its arguments. This is not the case in most other languages. This feature is called prefix notation, pre meaning before. Most languages use a combination of prefix, infix and postfix notations. e.g. sin(45), 6+5 and Count++ are all legitimate in some languages. In Lisp these expressions would be (sin 45), (+ 6 5) and (++ Count).

To create a Lisp program we nest lists within other lists. For example (+ (+ 3 5) (+ 5 7)) evaluates to 20.This is all the syntax that Lisp uses! One of the great things about this language is that the syntax is extremely simple and is consistently so. There are disadvantages however with this syntax. The main one of course, being the incredible number of brackets in Lisp programs. Most Lisp editors are able to automatically control brackets for you, or at least tell you when you've missed one.

To define a function in Lisp you use the defun function. This function is used in the same way as any other function within a list. For example (defun cube(x) (* x x x)). We could use this function to find the cube of 3, (cube 3) which of course would evaluate to 27.

```
(defun listlength (list)
        (if (null list)
                0
                (+ listlength (rest list)) 1)))
```

*Fig 1.2.72*

The function listlength above calculates the number of items in a list using recursion. The expression (listlength (9 8 23 6 4)) returns 5, as there are 5 items in the argument list. The function rest returns a list minus its first item e.g. (rest (3 7 2)) evaluates to the list 7 2.

## CLASS ACTIVTY

**Activity: listlength function**
Can you work out how the listlength function above works? What are the arguments for the null and if functions? What is the base case?
Work through the evaluation of the list (listlength (1 13 12 5 6 7)).

## EXERCISE 1.2.5 – SHORT ANSWER

1. In terms of logical programming, what is the name of a database that contains facts and rules?

   A. Knowledge base
   B. Logical base
   C. Fact base
   D. Query language

2. For logical programming languages processing is performed by the:

   A. control mechanism
   B. goal fulfillment
   C. inference engine
   D. knowledge base

3. Algorithms based on the sequence, selection, repetition control structures are the basis of which programming paradigm?

   A. Object oriented
   B. Imperative
   C. Logic
   D. Functional

4. Theobald is testing a theory and has started by assuming the theory is true. Which strategy is Theo most likely using to resolve this goal?

   A. Forward chaining
   B. Goal fulfillment
   C. Backward chaining
   D. Sequential testing

5. The following is a fact written using Prolog syntax. jump(horse,fence). Which part of the syntax is called a predicate?

   A. jump
   B. fence
   C. horse
   D. The entire fact

6. The OOP term Class can best be described as a category of:

   A. attributes
   B. objects
   C. methods
   D. processes

7. Which of the following means the original data cannot be changed once created?

   A. assignment
   B. side effect
   C. recursion
   D. immutability

8. A small inline function that will not be reused, in functional paradigm terms, is known as a:

   A. Lambda function
   B. First-class function
   C. Higher-order function
   D. Object method

9. Which paradigm evolved most directly from mathematical theory?

   A. Object oriented
   B. Imperative
   C. Logic
   D. Functional

10. Evaluate the Lisp expression (+ (* 2 3 5) (+ 6 5 3 1)).

    A. 25
    B. 45
    C. 150
    D. 450

## EXERCISE 1.2.5 - SHORT ANSWER

11. Briefly outline the essential defining features of each of the programming paradigms studied.
    - Object oriented
    - Imperative
    - Logic
    - Functional

12. Lisp uses prefix notation for all its functions. There is also infix and postfix notation in addition to prefix notation. Research and provide examples of functions in different programming language that use prefix, infix and postfix notation.

The relationships between family members can be represented as a knowledge base. Examine the relationships existing in the following scenario. Use this scenario to answer questions 13, 14 and 15.

- Bob has three sisters, Mary, Alice and Louise.
- Bob's parents are Ted and Margo.
- Margo's parents are John and Sue.
- Margo has two brothers, Mike and Tom.
- Tom's parents are Fred and Wilma.
- Mike has two sons, Dave and Paul.

13. Develop a list of facts using Prolog syntax, to represent each of the above relationships. For example sister(bob,mary) means Mary is Bob's sister. Make sure to include facts in regard to the sex of each family member eg. Female(Margo) and Male(Bob).

14. Family relationships adhere to strict rules. For example, to be somebody's grandmother you must be female and your child must be the parent of the grandchild.

    This could be expressed as the Prolog rule:
    grandmother(A,B):-female(A),mother(A,C),parent(C,B).

    Write rules for each of the following relationships:

    - Mother
    - Father
    - Sister
    - Brother
    - Cousin
    - Aunt
    - Uncle
    - Parent
    - Grandmother
    - Grandfather

15. Use your knowledge base to determine the following:

    Use a backward chaining strategy to confirm that Margo is Dave's aunty.

    Use a forward chaining strategy to find all of John's grandchildren.